



ISMAGi.

Institut Supérieur de Management
d'Administration et de Génie Informatique
المعهد العالي للتدبير والإدارة والهندسة المعلوماتية

Institut Supérieur de Management d'Administration et
de Génie Informatique

Projet de Fin d'Études

**Un simulateur de constellation Edge AI
orienté mission pour la surveillance
maritime sur CubeSats**

Réalisé par : KINANI NKAYA Franck

Soutenu devant le jury composé de :

**Présidente et
Examinatrice**

ACODAD Yosra

Encadran(e)

MINAOUI Khalid

Année universitaire **2025 - 2026**



ISMAGi.
**ENGINEERING
SCHOOL**

Remerciements

Au terme de ce stage de fin d'études, nous tenons à exprimer notre sincère gratitude envers l'ensemble des personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Nos remerciements les plus chaleureux s'adressent tout d'abord au **Dr. Kayyali Mohamed**, notre encadrant professionnel au sein de KSF Space Foundation, pour la confiance qu'il nous a accordé tout au long de ce stage, pour la richesse de ses conseils techniques, sa disponibilité constante et son accompagnement dans l'orientation de ce projet vers une contribution scientifique aboutie.

Nous exprimons également notre profonde reconnaissance à **Minaoui Khalid**, notre encadrant académique, pour son suivi rigoureux, ses remarques pertinentes et son soutien méthodologique qui ont permis de structurer ce rapport conformément aux exigences scientifiques et académiques attendues.

Nous remercions l'ensemble du corps professoral et administratif de l'iSMAGi, et plus particulièrement le pôle Engineering School, pour la qualité de la formation dispensée durant ces quatre années, qui nous a permis d'acquérir les compétences techniques et méthodologiques nécessaires à la conduite de ce projet.

Nos remerciements s'adressent aussi à toute l'équipe de KSF Space Foundation, et en particulier à **Madame Najlae**, pour leur accueil, leur esprit collaboratif et les échanges constructifs qui ont enrichi notre compréhension des enjeux opérationnels liés à la surveillance maritime spatiale.

Enfin, nous souhaitons remercier nos proches et nos camarades de promotion pour leur soutien constant et leurs encouragements tout au long de cette période de stage.

Sommaire

Remerciements.....	...
Liste des abréviations.....	...
Introduction générale.....	1
Chapitre 1 — Contexte et problématique.....	3
Chapitre 2 — État de l'art.....	8
Chapitre 3 — Architecture du système.....	14
Chapitre 4 — Méthodologie expérimentale.....	27
Chapitre 5 — Résultats.....	39
Chapitre 6 — Discussion.....	48
Conclusion générale.....	59
Bibliographie.....	64
Annexes.....	66
Table des matières.....	73

Liste des figures et liste des tableaux

Tableau 1 : Taxonomie des zones du filtre géospatial

Tableau 2 : Matrice de priorité tactique à deux dimensions

Tableau 3 : Formats de payload JSON produits par le sérialiseur

Tableau 4 : Décisions architecturales majeures et alternatives écartées

Tableau 5 : Jeux de données mobilisés dans ce travail

Tableau 6 : Hyperparamètres d'entraînement pour YOLOv8n (détecteur) et YOLOv8n-seg (segmenteur)

Tableau 7 : Distribution par zone des coordonnées synthétiques du manifeste de test

Tableau 8 : Configurations expérimentales évaluées

Tableau 9 : Méthodologie de construction de la vérité terrain pour TERR

Tableau 10 : Métriques de validation du détecteur sur la fenêtre finale d'entraînement

Tableau 11 : Étude d'ablation : stratégie d'initialisation du segmenteur

Tableau 12 : Impact de la quantification INT8 sur la taille des modèles et la précision

Tableau 13 : Métriques de performance embarquée

Tableau 14 : Résultats de validation du filtre géospatial de l'étage 1

Tableau 15 : Métriques à l'échelle de la mission pour les trois configurations

Tableau 16 : Synthèse des limites principales de l'étude

Tableau 17 : Feuille de route des travaux futurs, organisée par priorité

Liste des abréviations

Les abréviations et acronymes suivants sont utilisés dans ce rapport. Chaque terme est défini lors de sa première occurrence dans le corps du texte ; cette liste en propose un récapitulatif alphabétique.

AIS	Automatic Identification System (système d'identification automatique des navires)
API	Application Programming Interface (interface de programmation applicative)
ARM	Architecture de processeur Advanced RISC Machine
CFAR	Constant False Alarm Rate (détecteur à taux de fausse alarme constant)
CNN	Convolutional Neural Network (réseau de neurones convolutif)
COCO	Common Objects in Context (format standard d'annotation d'images)
CPU	Central Processing Unit (unité centrale de traitement)
CSV	Comma-Separated Values (format de fichier tabulaire)
DRR	Data Reduction Ratio (taux de réduction volumétrique des données)
EXIF	Exchangeable Image File Format (métadonnées standard des fichiers image)
GPS	Global Positioning System (système de positionnement par satellites)
GPU	Graphics Processing Unit (unité de traitement graphique)
GRD	Ground Range Detected (format de produit Sentinel-1 post-traité)
HC	HIGH/CRITICAL (catégorie d'événements de priorité tactique élevée)
HRSID	High-Resolution SAR Images Dataset (jeu de données SAR à haute résolution)

HSV	Hue, Saturation, Value (espace colorimétrique teinte-saturation-valeur)
IA	Intelligence Artificielle
ICEYE	Constellation commerciale de satellites SAR (nom propre du fournisseur)
INT8	Quantification entière sur 8 bits (format de compression de modèle)
JPEG	Joint Photographic Experts Group (format de compression d'image)
JSON	JavaScript Object Notation (format de données structurées)
KSF	KSF Space Foundation (organisme d'accueil du stage)
LEO	Low Earth Orbit (orbite terrestre basse)
ML	Machine Learning (apprentissage automatique)
MN	Mille(s) Nautique(s)
MRSSD	Multi-sensor Remote Sensing Ship Dataset (jeu de données de navires multi-capteurs)
NMS	Non-Maximum Suppression (suppression des détections redondantes)
OBC	On-Board Computer (calculateur de bord)
ONNX	Open Neural Network Exchange (format ouvert d'échange de modèles de réseaux de neurones)
POSIX	Portable Operating System Interface (norme d'interopérabilité des systèmes d'exploitation)
RAM	Random Access Memory (mémoire vive)
REST	Representational State Transfer (style d'architecture d'API web)
RVB	Rouge-Vert-Bleu (espace colorimétrique)
SAR	Synthetic Aperture Radar (radar à synthèse d'ouverture)

SSDD	SAR Ship Detection Dataset (jeu de données de détection de navires SAR)
TERR	Tactical Event Recall Rate (taux de rétention des événements tactiques)
TIES	Tactical Intelligence Efficiency Score (score d'efficacité du renseignement tactique)
VLIZ	Vlaams Instituut voor de Zee (Institut Marin de Flandre)
VOC	Visual Object Classes (format d'annotation Pascal VOC)
XML	eXtensible Markup Language (langage de balisage extensible)
YAML	YAML Ain't Markup Language (format de fichier de configuration)
YOLO	You Only Look Once (famille d'architectures de détection d'objets en temps réel)
ZEE	Zone Économique Exclusive

Introduction générale

La surveillance du domaine maritime constitue un enjeu stratégique majeur pour les États côtiers disposant de vastes zones économiques exclusives. Le Maroc, avec près de 1,1 million de kilomètres carrés de zone économique exclusive répartis entre façade atlantique et méditerranéenne, illustre parfaitement cette problématique : la surveillance exhaustive d'un tel territoire maritime par les seuls moyens de patrouille classiques représente une charge opérationnelle considérable, difficilement soutenable dans la durée.

L'imagerie satellitaire à synthèse d'ouverture radar (SAR — Synthetic Aperture Radar) offre une réponse technique prometteuse à ce défi. Insensible aux conditions météorologiques et à la luminosité, elle permet une détection fiable des navires à partir de leur signature radar, indépendamment de l'heure ou de la couverture nuageuse. L'émergence des constellations de nanosatellites de type CubeSat a considérablement démocratisé l'accès à ce type d'observation, réduisant les coûts de déploiement tout en multipliant les fréquences de revisite orbitale.

Cette opportunité technologique se heurte cependant à une contrainte physique incontournable : un satellite en orbite basse (LEO — Low Earth Orbit) n'est visible depuis une station de réception au sol que durant quelques minutes par orbite, la majeure partie du temps orbital se déroulant hors de portée de tout lien de communication direct. Cette fenêtre de contact limitée impose de repenser fondamentalement la chaîne de traitement de l'information : transmettre l'intégralité des images capturées est techniquement impossible, ce qui implique de traiter, filtrer et prioriser l'information directement à bord du satellite, avant toute transmission vers le sol.

C'est dans ce contexte que s'inscrit le présent travail, réalisé dans le cadre d'un stage de fin d'études au sein de KSF Space Foundation. L'objectif est de concevoir, implémenter et évaluer un pipeline

d'intelligence artificielle embarquée (Edge AI) capable de détecter des navires dans des images SAR, d'en évaluer la priorité tactique selon des critères géospatiaux, puis de transmettre uniquement l'information jugée pertinente, sous des contraintes matérielles strictes représentatives d'un ordinateur de bord (OBC — On-Board Computer) de CubeSat.

Au-delà de la seule performance de détection, ce travail interroge une problématique plus large, insuffisamment traitée dans la littérature existante : comment évaluer la pertinence opérationnelle d'un système embarqué de surveillance maritime, au-delà des métriques classiques d'apprentissage automatique ? Un modèle affichant une excellente précision de détection n'a que peu de valeur s'il ne parvient pas à transmettre l'information critique dans le temps imparti par la fenêtre de contact. C'est pour répondre à cette question que ce travail introduit un cadre d'évaluation orienté mission, combinant taux de rétention des événements tactiques et taux de réduction volumétrique des données.

Le présent rapport s'organise en six chapitres. Le premier chapitre présente le contexte opérationnel de la surveillance maritime par satellite et formalise la problématique scientifique et technique traitée. Le deuxième chapitre propose une revue de la littérature existante autour de quatre axes : la détection de navires en imagerie SAR, le déploiement de l'intelligence artificielle embarquée sur plateformes spatiales, les techniques de compression de modèles pour l'inférence sous contrainte, et les cadres d'évaluation orientés mission. Le troisième chapitre détaille l'architecture du système proposé, organisée en cinq étages de traitement conditionnels. Le quatrième chapitre décrit le protocole expérimental, les jeux de données utilisés et la méthodologie d'évaluation. Le cinquième chapitre présente les résultats obtenus, tant sur le plan de la performance des modèles que sur celui de l'efficacité opérationnelle du pipeline complet. Enfin, le sixième chapitre discute la portée et les limites de ces résultats, avant que la conclusion générale ne synthétise les contributions de ce travail et n'ouvre des perspectives pour une seconde phase de développement.

1. Chapitre 1 — Contexte et problématique

1.1 La surveillance maritime et l'opportunité des nanosatellites

La connaissance du domaine maritime (Maritime Domain Awareness) désigne la capacité d'un État côtier à surveiller en permanence l'activité des navires présents dans ses eaux territoriales et sa zone économique exclusive. Pour un pays comme le Maroc, dont la zone économique exclusive couvre environ 1,1 million de kilomètres carrés répartis entre l'océan Atlantique et la mer Méditerranée, cette surveillance représente un défi opérationnel majeur, notamment face aux enjeux de pêche illégale non déclarée et non réglementée, de trafic transfrontalier et de sécurité maritime en général.

L'imagerie radar à synthèse d'ouverture (SAR) constitue une réponse technique particulièrement adaptée à ce défi. Contrairement à l'imagerie optique classique, elle fonctionne indépendamment des conditions météorologiques et de l'éclairement solaire, ce qui lui permet de produire des cartes d'intensité de rétrodiffusion radar exploitables de jour comme de nuit, par tout temps. Les coques métalliques des navires produisent un signal de rétrodiffusion nettement plus élevé que celui de la surface marine environnante, ce qui permet leur détection fiable à des résolutions comprises entre 5 et 25 mètres selon les capteurs.

La constellation européenne Sentinel-1, opérée par l'Agence Spatiale Européenne dans le cadre du programme Copernicus, offre un accès gratuit à ce type d'imagerie sur les zones maritimes marocaines, avec une fréquence de revisite de l'ordre de six à douze jours. Le développement récent des constellations commerciales de nanosatellites de type CubeSat (Planet, Capella Space, ICEYE, Umbra, entre autres) a par ailleurs considérablement réduit les délais de revisite, ouvrant la voie à une surveillance quasi continue à moindre coût comparé aux satellites d'observation traditionnels.

1.2 La contrainte fondamentale : la fenêtre de contact orbital

Le déploiement de ces constellations de nanosatellites se heurte néanmoins à une contrainte physique incontournable, propre à l'orbite basse terrestre (LEO). Un satellite en orbite basse ne demeure visible depuis une station de réception au sol que durant une fenêtre de contact limitée — typiquement cinq à huit minutes — sur une période orbitale complète avoisinant quatre-vingt-quatorze minutes. Le reste du temps, le satellite se trouve hors de portée de toute liaison de communication directe avec le sol.

Cette contrainte a une implication directe sur l'architecture de traitement embarquée : un CubeSat produisant des tuiles d'image SAR d'environ un mégaoctet chacune accumule, sur une orbite complète, plusieurs gigaoctets de données brutes avant la prochaine fenêtre de contact. Avec un débit de liaison descendante typique en bande S de l'ordre d'un mégabit par seconde, la transmission de ce volume de données brutes saturerait intégralement la fenêtre de contact disponible, pour ne restituer au sol qu'une fraction infime des images capturées, et ce, avec un délai de plusieurs orbites.

Cette limitation structurelle impose de repenser fondamentalement la chaîne de traitement de l'information satellitaire. Plutôt que de transmettre l'intégralité des images capturées pour un traitement a posteriori au sol, il devient nécessaire d'effectuer un traitement intelligent directement à bord du satellite — détection, analyse, priorisation — de manière à ne transmettre que l'information tactiquement pertinente, sous une forme compacte, dans les limites du calculateur de bord disponible.

1.3 Formulation de la problématique

Cette contrainte de bande passante orbitale définit le problème de recherche central de ce travail, qui peut être formulé ainsi :

Comment un système embarqué d'intelligence artificielle peut-il sélectionner, compresser et prioriser de manière autonome le sous-ensemble d'images SAR tactiquement pertinent pour

transmission, dans les limites mémoire et calculatoires d'un calculateur de bord de CubeSat, sans intervention humaine entre la capture et la transmission ?

Ce problème se décompose naturellement en trois sous-problèmes distincts, individuellement traitables, mais conjointement contraints par les ressources disponibles à bord.

1.3.1 Détection sous contrainte de ressources

Les modèles classiques de détection de navires en imagerie SAR — qu'il s'agisse d'architectures à deux étages telles que Faster R-CNN, d'architectures à base de transformeurs, ou même de configurations YOLOv8 en pleine précision — nécessitent généralement une accélération matérielle par GPU ou une empreinte mémoire dépassant le gigaoctet pour une inférence en temps réel. Or, les calculateurs de bord des CubeSats de classe 3U à 6U (par exemple les architectures iMX8 ou ARM Cortex-A53) disposent typiquement d'un seul cœur de calcul, d'une mémoire vive comprise entre 256 mégaoctets et un gigaoctet, et d'aucune unité de traitement neuronal dédiée. Un modèle de détection déployable doit donc être compressé pour fonctionner dans ces limites, sans dégradation catastrophique de sa précision.

1.3.2 Priorisation tactique de l'information

Toutes les détections de navires n'ont pas la même valeur opérationnelle. Un regroupement de plusieurs navires dans les eaux territoriales, à proximité immédiate des côtes, constitue un événement de surveillance de priorité nettement plus élevée qu'un navire isolé transitant en haute mer internationale. Un système qui transmettrait l'ensemble des détections avec un traitement uniforme gaspillerait une part significative de la bande passante disponible sur des observations de faible valeur, au risque de ne pas restituer à temps les alertes réellement critiques. Un moteur de priorisation tenant compte du contexte géospatial est donc

nécessaire pour hiérarchiser les événements détectés et allouer la bande passante en conséquence.

1.3.3 Évaluation à l'échelle de la mission

Les métriques classiques d'apprentissage automatique — précision moyenne (mAP), précision, rappel — mesurent la performance d'un modèle sur un jeu de données de test, mais ne renseignent ni sur la criticité opérationnelle des détections manquées, ni sur l'adéquation du volume de données produit avec le budget de bande passante de la fenêtre de contact. Un cadre d'évaluation capable de quantifier conjointement la rétention de l'information tactique et l'efficacité de la réduction volumétrique des données est nécessaire pour comparer objectivement différentes configurations d'un même pipeline embarqué.

1.4 Objectifs du travail

Ce travail répond à ces trois sous-problèmes par la conception d'un système unifié, structuré en une chaîne de décision à cinq étages : filtrage géospatial, détection des navires, moteur de priorité tactique, segmentation conditionnelle, et sérialisation de la télémétrie. L'ensemble du système est évalué sous des contraintes matérielles simulant fidèlement un calculateur de bord de CubeSat — un seul cœur de calcul et un gigaoctet de mémoire vive — et sa performance est mesurée à l'aide d'un cadre d'évaluation orienté mission, introduit dans le cadre de ce travail, combinant taux de rétention des événements tactiques et taux de réduction volumétrique des données.

Les objectifs spécifiques poursuivis dans ce rapport sont les suivants :

- concevoir une architecture de traitement conditionnelle où le coût de calcul de chaque étage est proportionné à la valeur tactique de l'information traitée ;

- entraîner et compresser des modèles de détection et de segmentation de navires adaptés à l'imagerie SAR, tout en respectant les contraintes mémoire d'un ordinateur embarqué ;
- évaluer l'impact de la quantification des modèles (passage de la simple à la précision entière sur 8 bits) sur la performance opérationnelle du système, au-delà des seules métriques d'apprentissage automatique ;
- proposer et valider un cadre de métriques orienté mission, permettant de comparer différentes configurations du pipeline selon des critères pertinents pour une mission réelle de surveillance maritime.

2. Chapitre 2 — État de l'art

Ce chapitre propose une revue de la littérature scientifique organisée autour de quatre axes complémentaires, correspondant aux dimensions techniques et méthodologiques mobilisées dans ce travail : la détection de navires en imagerie SAR, le déploiement de l'intelligence artificielle embarquée sur plateformes spatiales, les techniques de compression de modèles pour l'inférence sous contrainte, et les cadres d'évaluation orientés mission pour les systèmes d'observation terrestre autonomes. Pour chaque axe, les limites de l'état de l'art existant sont identifiées, motivant les choix de conception présentés au chapitre suivant.

2.1 Détection de navires en imagerie SAR

La détection de navires en imagerie radar à synthèse d'ouverture fait l'objet de recherches actives depuis plus de deux décennies. Les approches historiques reposaient sur des détecteurs à taux de fausse alarme constant (CFAR), exploitant le contraste statistique entre le signal de retour des coques métalliques et le fouillis de la surface océanique environnante. Ces méthodes, bien qu'efficaces par mer calme, se dégradent nettement en présence d'états de mer élevés ou de regroupements denses de navires.

L'essor de l'apprentissage profond a profondément transformé ce domaine, porté par la disponibilité croissante de jeux de données annotés tels que SSDD (Li et al., 2017), SAR-Ship (Wang et al., 2019), OpenSARShip (Hu et al., 2020) et MRSSD. Les architectures à deux étages, notamment les variantes de Faster R-CNN, ont démontré de solides performances sur ces benchmarks — Li et al. (2022) rapportent une précision moyenne (mAP@0.5) de 0,921 sur SSDD — mais leur latence d'inférence, comprise entre 600 et 1200 millisecondes sur processeur central, les rend incompatibles avec les exigences de traitement embarqué en temps réel.

Les architectures à un seul étage, en particulier la famille YOLO, offrent un compromis nettement plus favorable entre vitesse et précision. Zhao et al. (2022) démontrent que YOLOv5s atteint une mAP@0.5 de 0,934 sur le jeu de données HRSID avec une latence d'inférence inférieure à 40 millisecondes sur GPU. Huang et al. (2023) appliquent YOLOv8n au jeu de données MRSSD, rapportant une mAP@0.5 de 0,941, la variante nano de cette architecture (3,2 millions de paramètres) permettant une inférence en temps réel sur processeurs ARM. Le présent travail s'appuie sur cette architecture, affinée sur l'ensemble complet du jeu d'entraînement MRSSD avec une politique d'augmentation adaptée aux spécificités de l'imagerie SAR, atteignant une mAP@0.5 de 0,9474.

Une limite persistante de cette littérature réside dans le protocole d'évaluation employé : la quasi-totalité des travaux publiés rapportent des métriques de précision moyenne, précision et rappel sur des ensembles de test issus du même jeu de données que celui utilisé pour l'entraînement. Ce protocole mesure la capacité de généralisation intra-domaine, mais n'apporte aucune réponse à deux questions pourtant centrales pour une application opérationnelle de surveillance maritime : quelle proportion des événements tactiquement significatifs le pipeline complet parvient-il à identifier correctement sous une logique de priorisation géospatiale, et quel est le coût de transmission résultant si la sortie du pipeline se substitue à la transmission de l'image brute ?

2.2 Intelligence artificielle embarquée sur plateformes spatiales

L'architecture dominante de l'observation terrestre par satellite a longtemps séparé l'acquisition des données, réalisée à bord, de leur traitement, effectué au sol après transmission de l'image brute. Ce modèle, adapté aux applications non critiques en termes de délai, s'avère inadapté à la surveillance maritime, où la position des navires évolue significativement en quelques minutes, et où les fenêtres de contact limitent les opportunités de transmission à quelques passages quotidiens.

Le concept d'inférence embarquée — l'exécution de modèles de réseaux de neurones directement sur le calculateur de bord, avant transmission — a été démontré pour la première fois en conditions opérationnelles par la mission PhiSat-1 de l'Agence Spatiale Européenne. Giuffrida et al. (2021) décrivent le déploiement d'un réseau de neurones convolutif de détection nuageuse sur l'unité de traitement vectoriel Intel Myriad X intégrée au calculateur de bord de PhiSat-1, contrainte à quatre gigaoctets de mémoire vive, atteignant une précision de 92 % en orbite. Cette démonstration constitue une référence majeure en matière de faisabilité de l'IA embarquée, mais elle porte sur une tâche de classification binaire mono-tâche, non sur un pipeline complet de détection et de segmentation.

Une étude systématique de l'Agence Spatiale Européenne, menée par Furano et al. (2022), dresse un panorama des calculateurs de bord compatibles avec l'IA disponibles pour les missions de petits satellites, concluant que la famille Nvidia Jetson représente la frontière de performance actuelle, tandis que les processeurs de classe ARM Cortex-A53/A72 (256 mégaoctets à un gigaoctet de mémoire vive, sans GPU) constituent la cible de déploiement réaliste pour les CubeSats de forme 3U. Le présent travail cible précisément cette seconde classe de matériel, avec une contrainte mémoire d'un gigaoctet sur processeur central uniquement, plus restrictive que celle rencontrée dans la majorité des expérimentations publiées d'IA embarquée en environnement spatial.

Un corpus de travaux plus restreint s'est intéressé à l'interaction entre géométrie orbitale et ordonnancement de l'inférence embarquée. Denby et Lucia (2020) introduisent le concept de calcul en périphérie orbitale (« orbital edge computing »), démontrant qu'un ordonnancement conscient du cycle orbital peut réduire jusqu'à 60 % les calculs redondants, comparé à un traitement continu. Leur système expérimental, ciblant le filtrage d'images optiques, ne comporte cependant aucun modèle de détection ni aucune quantification de la réduction de bande passante obtenue — deux dimensions au cœur du présent travail.

2.3 Compression de modèles pour l'inférence sous contrainte

La quantification post-entraînement s'est imposée comme l'approche standard pour réduire l'empreinte mémoire et la latence d'inférence des modèles d'apprentissage profond sur matériel contraint. Deux stratégies principales coexistent : la quantification statique, où les échelles d'activation sont calculées à partir d'un jeu de données de calibration fixé avant déploiement, et la quantification dynamique, où la quantification des poids est effectuée hors ligne tandis que les échelles d'activation sont calculées au moment de l'inférence, à partir de la distribution réelle des données d'entrée.

Pour les modèles légers de la famille YOLO, les études publiées rapportent de manière constante des réductions de taille de l'ordre de 40 à 50 % en quantification INT8, pour une dégradation de précision moyenne comprise entre 0,5 et 2,0 points. Le présent travail observe une réduction de taille de 46,3 % avec une perte de mAP@0.5 de 0,84 % pour le détecteur, résultat cohérent avec la littérature existante. Le résultat le plus significatif — à notre connaissance non explicitement documenté pour la surveillance maritime SAR — est que la configuration INT8 ne provoque aucune défaillance systématique sur la classification des événements de priorité HIGH ou CRITICAL, suggérant que l'information discriminante pertinente pour la priorisation tactique est préservée sous quantification pour cette classe d'imagerie et d'architecture de modèle.

Des approches alternatives de compression — distillation de connaissances, recherche d'architecture neuronale, élagage structuré — ont également été explorées pour la détection SAR. Liu et al. (2022) appliquent la distillation de connaissances d'un modèle enseignant ResNet-101 vers un modèle élève MobileNetV3 sur le jeu de données SSDD, atteignant 89 % de la performance de l'enseignant pour 12 % de son nombre de paramètres. Ces approches, bien que potentiellement plus efficaces, nécessitent des infrastructures d'entraînement additionnelles, ce qui a motivé le choix de la quantification dynamique post-entraînement

dans ce travail, pour sa simplicité opérationnelle et son coût de précision négligeable sur le jeu de données cible.

2.4 Cadres d'évaluation orientés mission

L'évaluation des systèmes d'intelligence artificielle embarquée pour l'observation terrestre révèle une tension fondamentale entre les métriques usuelles de la communauté de l'apprentissage automatique et celles réellement pertinentes pour une mission opérationnelle. Les métriques classiques traitent toutes les erreurs de détection de manière symétrique : un navire manqué en haute mer est pénalisé de façon identique à un navire manqué en zone territoriale sensible, alors que leur portée opérationnelle diffère radicalement.

Cette limite a été reconnue dans d'autres domaines de l'IA embarquée pour systèmes autonomes. Dans le domaine de la conduite autonome, le défi Waymo Open Dataset a introduit des métriques pondérées par scénario, pénalisant plus sévèrement les événements de quasi-collision que les cas de détection triviaux. À notre connaissance, aucun cadre d'évaluation pondéré par la mission équivalent n'a été proposé spécifiquement pour le domaine de la surveillance maritime par imagerie SAR.

La réduction de bande passante a par ailleurs été quantifiée dans la littérature sur l'IA embarquée sous diverses appellations. Denby et Lucia rapportent un « taux de rejet de données » pour leur système de filtrage optique ; Giuffrida et al. (2021) rapportent la fraction de tuiles classées comme dépourvues de nuages comme métrique d'efficacité de débit pour PhiSat-1. Ces métriques capturent une dimension du problème de bande passante — le volume de données écarté — sans toutefois renseigner sur la pertinence tactique des données effectivement conservées.

Le présent travail introduit un score composite, dénommé TIES (Tactical Intelligence Efficiency Score), défini comme le produit de deux métriques : le taux de rétention des événements tactiques (TERR) et le

taux de réduction volumétrique des données (DRR). La formulation multiplicative encode une contrainte opérationnelle simple mais structurante : un système défaillant sur l'une ou l'autre de ces deux dimensions — rétention tactique nulle, ou absence de compression — ne présente aucune valeur de mission, quelle que soit sa performance sur l'autre dimension. Cette propriété distingue TIES d'une combinaison additive pondérée, qui autoriserait une compensation partielle entre les deux dimensions.

2.5 Positionnement du présent travail

La revue de littérature qui précède permet d'identifier quatre limites spécifiques de l'état de l'art existant, auxquelles ce travail apporte une réponse. Premièrement, l'absence d'un cadre d'évaluation orienté mission pour les pipelines de détection maritime SAR combinant rétention tactique et efficacité de bande passante. Deuxièmement, l'absence de démonstration publiée d'un pipeline conditionnel complet — détection, segmentation, priorisation géospatiale — fonctionnant sous une contrainte mémoire d'un gigaoctet sur processeur central uniquement. Troisièmement, l'absence d'une simulation du cycle orbital complet couplée à un pipeline de détection pour l'évaluation de la performance à l'échelle de la mission. Quatrièmement, l'absence de mesure explicite de l'impact de la quantification INT8 sur la rétention des événements tactiques prioritaires, par opposition à une dégradation uniforme de l'ensemble des détections.

Il convient de souligner que ce travail ne revendique pas une supériorité sur l'état de l'art en matière de métriques de détection pures : la mAP@0.5 de 0,9474 obtenue est compétitive avec les travaux récents fondés sur YOLOv8, sans toutefois dépasser l'état de l'art sur aucun des benchmarks considérés. La contribution de ce travail se situe davantage au niveau de l'architecture système et de la méthodologie d'évaluation qu'à celui du modèle pris isolément.

3. Chapitre 3 — Architecture du système

Ce chapitre décrit en détail l'architecture du système développé au cours de ce stage. La conception repose sur un principe organisateur unique, présenté en section 3.1, puis déclinée en cinq étages de traitement successifs (sections 3.2 à 3.6), avant une présentation de l'environnement de simulation Docker (section 3.7), une synthèse des choix architecturaux majeurs (section 3.8) et des mécanismes de robustesse mis en œuvre (section 3.9).

3.1 Philosophie de conception : chaîne de décision plutôt que traitement par lot

L'architecture repose sur un principe organisateur central : le coût de calcul doit être proportionnel à la valeur tactique de l'information traitée. Un traitement par lot classique applique uniformément l'ensemble des étages de traitement à chaque image, puis écarte a posteriori les résultats non pertinents. Une chaîne de décision, à l'inverse, conditionne chaque étage à une validation binaire de l'étage précédent — le traitement ne se poursuit que si l'étage précédent en confirme la pertinence. Cette asymétrie constitue le choix de conception le plus structurant du pipeline développé, et le mécanisme principal de son efficacité en matière de réduction de bande passante.

La conséquence directe de ce principe est un modèle de latence par morceaux. En notant $\Delta t_{\text{pipeline}}$ le temps total de traitement d'une tuile SAR unique, le pipeline se résout selon quatre branches d'exécution distinctes, selon les conditions rencontrées dans l'image :

Condition 1 (image hors zone de surveillance) : $\Delta t = \Delta t_{\text{géo}}$

Condition 2 (mer vide, aucun navire détecté) : $\Delta t = \Delta t_{\text{géo}} + \Delta t_{\text{dét}}$

Condition 3 (priorité LOW ou MEDIUM) : $\Delta t = \Delta t_{\text{géo}} + \Delta t_{\text{dét}} + \Delta t_{\text{priorité}} + \Delta t_{\text{sérial}}$

Condition 4 (priorité HIGH ou CRITICAL) : $\Delta t = \Delta t_{\text{géo}} + \Delta t_{\text{dét}} + \Delta t_{\text{priorité}} + \Delta t_{\text{seg}} + \Delta t_{\text{sérial}}$

L'étage le plus coûteux en calcul — la segmentation d'instance (Δt_{seg}) — n'est actif qu'en Condition 4. Dans l'évaluation expérimentale conduite sur le jeu de test MRSSD, avec la distribution de coordonnées Z1/Z2/Z3 décrite au chapitre 4, environ 35% des tuiles traitées déclenchent la Condition 4, ce qui signifie que la segmentation ne s'exécute que sur approximativement une tuile sur trois. Les 65% restants sont résolus dès les Conditions 1 à 3, avec un surcoût de segmentation négligeable.

3.2 Étage 1 — Filtre géospatial (geo_filter.py)

Le filtre géospatial constitue l'étage le plus économe en calcul du pipeline, et simultanément celui présentant le meilleur rapport bénéfice/coût. Il opère sur une entrée scalaire unique — les coordonnées GPS (latitude, longitude) associées à chaque tuile SAR — et effectue une requête d'intersection de polygones en complexité $O(1)$ contre un ensemble de fichiers de limites GeoJSON. Aucune donnée image n'est manipulée à ce stade. Si la requête retourne Z0 (non-maritime), la tuile est immédiatement écartée et aucune inférence en aval n'est déclenchée. Dans le scénario de déploiement visé, ce mécanisme élimine environ 80% des tuiles capturées qui survolent des zones terrestres ou océaniques hors surveillance lors d'un passage orbital LEO.

La taxonomie de zones utilisée par le filtre est dérivée de Marine Regions v12 (Institut Marin de Flandre, VLIZ), le jeu de données mondial de référence pour les zones économiques exclusives. Trois fichiers GeoJSON de production sont chargés à l'initialisation, couvrant la ZEE marocaine, la ZEE du Sahara occidental, et les eaux territoriales marocaines des 12 milles nautiques. Pour respecter la contrainte de 1 gigaoctet de RAM de l'environnement OBC simulé, l'ensemble des polygones est simplifié à 90-95% de réduction de sommets à l'aide de la fonction `geometry.simplify()` de la bibliothèque Shapely, avec l'option `preserve_topology=True`. La taxonomie de zones résultante est présentée au Tableau 1.

Tableau 1 : Taxonomie des zones du filtre géospatial

Zone	Désignation	Définition	Sensibilité	Action du pipeline
Z0	Non-maritime	Hors de toute limite de ZEE	0	Rejet immédiat — aucune inférence
Z1	Eaux territoriales (12 MN)	Dans les 12 milles nautiques de la ligne de base marocaine	1 (max.)	Pipeline complet — segmentation dès 1 navire
Z2	Zone économique exclusive	ZEE au-delà de 12 MN (Maroc + Sahara occidental)	2	Pipeline complet — segmentation dès 2 navires
Z3	Haute mer internationale	Au-delà de la ZEE — eaux internationales	3 (min.)	Détection seule — segmentation dès 5 navires

Source : élaboration propre, d'après Marine Regions v12

La sensibilité de zone présentée au Tableau 1 gouverne directement la matrice de priorité du moteur tactique, décrite en section 3.4. Z0 est la seule condition qui interrompt inconditionnellement l'exécution du pipeline.

Justification du choix d'une recherche géospatiale plutôt qu'un classifieur de scène appris. Une alternative envisagée durant la phase de conception consistait en un classifieur de scène appris, entraîné pour distinguer les images maritimes des images non maritimes. Cette approche a été écartée pour trois raisons. Premièrement, les imageries SAR et optique présentent des signatures spectrales fondamentalement différentes — un classifieur entraîné sur données optiques ne se transfère pas à l'imagerie SAR sans adaptation de domaine, ce qui nécessiterait un jeu de données annoté supplémentaire. Deuxièmement, un classifieur appris ajoute une latence d'inférence sur chaque tuile, alors que la recherche GeoJSON est en complexité $O(1)$ et déterministe. Troisièmement, et de manière plus critique, un classifieur appris est de nature stochastique — sa sortie dépend des poids du modèle et peut dériver sous décalage de distribution. Une intersection de polygones offre au contraire une garantie

mathématique : si la coordonnée GPS se trouve à l'intérieur du polygone de la ZEE, la tuile est maritime.

3.3 Étape 2 — Détection des navires (detector.py)

La détection des navires est assurée par un réseau YOLOv8n (Ultralytics, version 8.3.110) affiné sur le jeu de données iVision-MRSSD et exporté au format ONNX pour une inférence exclusivement CPU via ONNX Runtime. L'architecture YOLOv8n a été retenue pour trois raisons : son nombre de paramètres (3,2 millions) s'inscrit dans le budget de stockage de modèle de 10 mégaoctets par module après quantification INT8 (3,36 mégaoctets mesurés) ; sa tête de détection à passage unique est compatible avec l'opset ONNX 17 ; et sa performance sur les benchmarks de détection de navires SAR, examinés au chapitre 2, se révèle compétitive avec des architectures plus lourdes tout en restant déployable sur processeurs ARM sans accélérateur dédié.

Prétraitement spécifique à l'imagerie SAR. L'imagerie SAR se présente sous forme de cartes d'intensité monocanal encodant l'énergie de rétrodiffusion radar depuis la surface océanique. Les coques métalliques des navires produisent des réflexions spéculaires intenses, apparaissant comme des cibles ponctuelles de forte intensité entourées de fouillis océanique. Le pipeline de prétraitement applique les transformations suivantes avant de transmettre une tuile au détecteur :

- l'image d'intensité SAR en niveaux de gris est répliquée sur trois canaux pour satisfaire le format d'entrée RVB attendu par YOLOv8n — aucune information de couleur n'est ajoutée, les trois canaux étant des copies identiques de la carte d'intensité ;
- les valeurs de pixels sont normalisées dans l'intervalle [0, 1] par division par 255 ;
- l'image est redimensionnée à 640×640 pixels par interpolation bilinéaire, en préservant le ratio d'aspect par remplissage à zéro (zero-padding).

Les augmentations spécifiques à l'imagerie SAR appliquées durant l'entraînement sont les suivantes : HSV-H = 0,0 et HSV-S = 0,0 (les décalages de teinte et de saturation sont désactivés, l'imagerie SAR étant fondée sur l'intensité et non sur la couleur) ; HSV-V = 0,3 (variation de luminosité simulant les fluctuations d'intensité radar) ; Mosaic = 1,0 (élément déterminant pour la détection de petits navires dans de vastes étendues marines) ; Flip-V = 0,3 et Flip-H = 0,5 (invariance à la direction orbitale de passage). Ces choix d'augmentation ont été validés par la convergence de l'entraînement décrite au chapitre 5.

Paramètres d'inférence : seuil de confiance = 0,45, seuil IoU de suppression non maximale (NMS) = 0,45. La session ONNX Runtime est configurée avec `intra_op_num_threads = 1`, `inter_op_num_threads = 1`, et `GraphOptimizationLevel = ORT_ENABLE_BASIC`, afin de respecter la contrainte mono-cœur de l'environnement OBC simulé.

Sortie du détecteur : une liste de boîtes englobantes détectées en coordonnées normalisées [x1, y1, x2, y2], accompagnées de scores de confiance et d'identifiants de classe. Dans la configuration actuelle du modèle (classe 0 : navire), toutes les détections sont sémantiquement équivalentes ; la discrimination multi-classes est envisagée comme travail futur (chapitre 6). Si aucune détection n'est retournée, le pipeline s'interrompt immédiatement sans solliciter le moteur de priorité ni le segmenteur — c'est la Condition 2 du modèle de latence par morceaux présenté en section 3.1.

3.4 Étage 3 — Moteur de priorité tactique (priority_engine.py)

Le moteur de priorité transforme la sortie brute de détection — un nombre de navires et un niveau de sensibilité de zone — en une classe de priorité tactique discrète, qui détermine la branche de traitement empruntée par le pipeline. Il met en œuvre une matrice de correspondance déterministe à deux dimensions, indexée par la sensibilité de zone (1, 2 ou 3) et le regroupement du nombre de navires (isolé : $n = 1$,

groupe : $2 \leq n \leq 4$, rassemblement : $n > 4$). Les classes de priorité résultantes sont LOW, MEDIUM, HIGH et CRITICAL. La matrice complète est présentée au Tableau 2.

Tableau 2 : Matrice de priorité tactique à deux dimensions

Zone / Nombre de navires	Isolé (= 1)	Groupe (2 - 4)	Rassemblement (> 4)
Z1 — Eaux territoriales (sens. 1)	MEDIUM	HIGH	CRITICAL
Z2 — ZEE (sens. 2)	LOW	MEDIUM	HIGH
Z3 — Haute mer (sens. 3)	LOW	LOW	MEDIUM

Source : élaboration propre

Les cellules ombrées du Tableau 2 (HIGH, CRITICAL) déclenchent la segmentation conditionnelle décrite en section 3.5. La matrice encode la doctrine opérationnelle selon laquelle un navire isolé en eaux territoriales (Z1) présente une signification tactique supérieure à un groupe de navires en haute mer internationale (Z3).

Justification de la conception à deux axes. Un système à axe unique fondé uniquement sur le nombre de navires classerait de manière identique un navire isolé en eaux territoriales et un navire isolé en haute mer internationale, ignorant le contexte opérationnel pourtant radicalement différent. Un système à axe unique fondé uniquement sur la sensibilité de zone échouerait à distinguer une tuile Z1 vide d'un rassemblement de navires dans cette même zone. La conception à deux axes capture simultanément les deux dimensions de la signification tactique, et ne nécessite que deux recherches par tuile — une complexité $O(1)$.

Les événements HIGH et CRITICAL déclenchent deux actions en aval : premièrement, la segmentation est invoquée à l'étage 4 ; deuxièmement, le payload résultant est acheminé vers une transmission au Format B

(liaison descendante prioritaire et immédiate). Les événements LOW et MEDIUM contournent entièrement la segmentation et sont acheminés vers le Format A (liaison descendante en lot, différée). Le drapeau booléen `triggers_segmentation`, retourné par le moteur de priorité, constitue le signal unique consommé par l'orchestrateur pour effectuer cet aiguillage.

3.5 Étage 4 — Segmentation d'instance conditionnelle (`segmenter.py`)

La segmentation d'instance est assurée par un réseau YOLOv8n-seg affiné sur le jeu de données SSDD_coco et exporté au format ONNX INT8. Le segmenteur n'est activé que lorsque le moteur de priorité positionne `triggers_segmentation = True`, c'est-à-dire exclusivement pour les événements HIGH et CRITICAL. Cette activation conditionnelle constitue l'argument architectural en faveur de deux modèles distincts, par opposition à un modèle unique multi-tâches : un réseau YOLOv8n-seg appliqué systématiquement à chaque tuile entraînerait un surcoût constant d'environ 45% de FLOPs supplémentaires (12,6 contre 8,7 GFLOPs à résolution 640 pixels). Appliquée conditionnellement, à un taux observé d'environ 35% des tuiles traitées, l'architecture conditionnelle à deux modèles réduit les FLOPs cumulés de segmentation d'environ 65% par rapport à une application systématique.

Stratégie d'initialisation à chaud. Le segmenteur a été initialisé à partir des poids de l'ossature du détecteur entraîné (`yolov8n_mrssd_v1/weights/last.pt`), plutôt qu'à partir des poids génériques `yolov8n-seg.pt` pré-entraînés sur COCO. Cette hypothèse d'initialisation à chaud spécifique au domaine SAR — selon laquelle une ossature déjà adaptée aux caractéristiques d'intensité SAR fournit une meilleure initialisation que des poids pré-entraînés sur données optiques COCO — est testée par une étude d'ablation décrite au chapitre 5. Deux configurations sont comparées : la Configuration A (initialisation à chaud depuis l'ossature du détecteur SAR) et la Configuration B (initialisation à chaud depuis les poids génériques COCO).

Sortie de segmentation par navire détecté : contour de masque de pixels (polygone simplifié, série de coordonnées normalisées [x, y]), surface estimée en pixels, angle d'orientation θ en degrés, et ratio d'aspect. L'ensemble de ces champs est inclus dans le payload Format B (section 3.6). Si le segmenteur échoue à produire un masque valide pour une détection donnée (par exemple en raison de la qualité de l'image), une boîte englobante de secours est utilisée, préservant la continuité du pipeline.

Protection anti-saturation mémoire. Pour protéger la limite de 1 gigaoctet de RAM face à des scénarios de densité de détection extrême (par exemple une zone portuaire dense produisant plus de 30 détections), l'orchestrateur applique un mécanisme de troncature pré-segmentation : si le nombre de détections dépasse un seuil configurable (par défaut 15, défini dans le fichier YAML de l'expérimentation), la liste est tronquée aux N plus grandes boîtes englobantes par surface avant d'être transmise au segmenteur. Ce seuil est configurable par expérimentation et doit être calibré selon les valeurs de RAM maximale mesurées pour le calculateur de bord ciblé.

3.6 Étape 5 — Sérialisation de la télémétrie (serializer.py)

Le sérialiseur compresse l'ensemble des sorties du pipeline en un payload de télémétrie JSON structuré, écrit sur le volume Docker partagé pour consommation par le tableau de bord de la station sol. Deux formats de payload sont définis, différant par leur contenu informationnel et leur urgence de transmission. Les deux formats remplacent entièrement la transmission d'image brute — la tuile SAR elle-même n'est jamais écrite en sortie. Le Tableau 3 résume les deux formats.

Tableau 3 : Formats de payload JSON produits par le sérialiseur

Format	Taille	Champs inclus	Mode de transmission
Format A	≈ 598 octets	identifiant d'événement, identifiant image, GPS, zone, priorité, nombre de navires, horodatage, latence détecteur, version du modèle, raison tactique	Différée — mise en tampon et transmise lors de la prochaine fenêtre de contact (événements LOW / MEDIUM)
Format B	≈ 994 octets	tous les champs du Format A, plus les boîtes englobantes par navire, le contour du masque de segmentation, la surface en pixels, l'orientation, le ratio d'aspect, la latence du segmenteur	Prioritaire — transmise sans mise en attente (événements HIGH / CRITICAL uniquement)

Source : élaboration propre, tailles mesurées sur payloads représentatifs

L'écart entre les tailles mesurées au Tableau 3 et les estimations de conception initiales (≈150 octets / ≈800 octets) s'explique par les champs de traçabilité (version du modèle, raison tactique, métadonnées de débogage) non comptabilisés dans le cahier des charges initial. Les deux tailles produisent des valeurs de DRR dépassant largement l'objectif de mission fixé à plus de 100x.

Mécanisme d'écriture atomique. Afin de prévenir les conditions de concurrence entre le sérialiseur (rédacteur) et le tableau de bord de la station sol (lecteur) lors d'accès simultanés au volume Docker partagé, l'ensemble des écritures de payload sont effectuées de façon atomique. Le sérialiseur écrit d'abord dans un fichier temporaire (payload.json.tmp), puis invoque `os.replace()` — une opération atomique au sens POSIX qui remplace le fichier cible final en un seul appel système, éliminant toute fenêtre de lecture partielle susceptible de provoquer une erreur `JSONDecodeError` côté tableau de bord. Ce mécanisme a été validé sous test de charge d'entrées/sorties concurrentes, avec un lecteur interrogeant le système à raison de 2000 lectures par seconde pendant que le

rédacteur effectuait 100 écritures successives d'un payload d'environ 10 kilooctets — aucune erreur de décodage n'a été observée.

3.7 Environnement de simulation Docker

Le système complet de bout en bout est conteneurisé selon une architecture Docker à deux services, simulant la séparation physique entre le calculateur de bord du CubeSat et le terminal de réception au sol.

cubesat-core (conteneur d'inférence). Ce conteneur exécute le pipeline complet à cinq étages sous des contraintes matérielles strictes : 1 cœur de calcul virtuel, plafond de 1 gigaoctet de RAM, `OMP_NUM_THREADS = 1`, `ONNXRUNTIME_NUM_THREADS = 1`. L'image de base est `python:3.10-slim` ; PyTorch est explicitement exclu de ce conteneur — l'inférence est assurée exclusivement via ONNX Runtime, qui ne nécessite aucun support GPU au niveau du framework. Cette configuration reflète la philosophie de dépendances minimales des calculateurs de bord CubeSat réels (par exemple iMX8, ARM Cortex-A53), qui ne peuvent héberger l'arborescence de dépendances de PyTorch, d'environ 1,5 gigaoctet.

ground-dashboard (conteneur de visualisation). Ce conteneur héberge une application Streamlit proposant trois vues : un moniteur en temps réel (état du pipeline, tuile SAR annotée avec ses détections, matrice de décision tactique), une vue d'analyse de mission (répartition des priorités, chronologie des latences, carte tactique maritime), et un flux d'événements (flux structuré de payloads JSON depuis le conteneur d'inférence). La communication entre les deux conteneurs est exclusivement assurée par un volume Docker partagé — le conteneur d'inférence écrit les payloads JSON et les fichiers d'état de battement de cœur (heartbeat) ; le tableau de bord les lit. Aucun socket réseau direct ni API n'est utilisé, ce qui simule le modèle de communication asynchrone de stockage-et-retransmission d'une véritable liaison descendante satellitaire.

Trois configurations expérimentales sont évaluées (détaillées au chapitre 4) : baseline (PyTorch pleine précision, hôte non contraint), `onnx_fp32`

(ONNX pleine précision, contraint sous Docker), et onnx_int8 (ONNX quantifié INT8, contraint sous Docker). La comparaison entre baseline et onnx_fp32 isole l'impact des contraintes matérielles indépendamment de la quantification. La comparaison entre onnx_fp32 et onnx_int8 isole l'impact de la quantification INT8 indépendamment des contraintes. Cette conception factorielle permet une attribution claire des variations de performance à leur cause spécifique.

3.8 Synthèse des décisions de conception majeures

Le Tableau 4 résume les cinq décisions architecturales principales, l'alternative envisagée dans chaque cas, et la justification quantitative ou qualitative du choix retenu. Ces décisions constituent, collectivement, la contribution architecturale de ce travail au-delà du simple recours à des modèles standards de détection et de segmentation.

Tableau 4 : Décisions architecturales majeures et alternatives écartées

Décision de conception	Choix retenu	Alternative écartée et justification
Environnement d'inférence	ONNX Runtime (indépendant de l'architecture, CPU uniquement)	TensorRT : réservé aux GPU NVIDIA, incompatible avec les calculateurs de bord ARM. TFLite : moins portable, spécifique à certains fournisseurs.
Filtre géospatial vs. classifieur de scène	Intersection de polygones GeoJSON — $O(1)$, déterministe	Classifieur appris : nécessite une adaptation de domaine optique-SAR, ajoute un coût d'inférence, comportement non déterministe.
Deux modèles vs. un modèle multi-tâches	Détecteur distinct + segmenteur conditionnel	YOLOv8n-seg appliqué à toutes les images : surcoût constant de +45% de FLOPs. À un taux d'activation de ~35% des événements HIGH/CRITICAL, l'approche à deux modèles économise ~65% des FLOPs cumulés de segmentation.

Stratégie de quantification	Quantification dynamique INT8 post-export	Quantification statique : nécessite un jeu de données de calibration, ajoute de la complexité au pipeline. La quantification dynamique suffit pour un premier déploiement avec une perte de mAP inférieure à 1%.
Format de payload	JSON structuré à deux variantes (A/B)	Transmission de l'image brute : contredit directement la contrainte de minimisation de la bande passante.

Source : élaboration propre

3.9 Conception défensive : validation des entrées et mécanismes de sécurité

Le fonctionnement en environnement autonome et non supervisé expose le pipeline à une classe de défaillances que les bancs d'essai en laboratoire ne révèlent généralement pas : tuiles image corrompues (erreurs de transmission, anomalies de capteur), métadonnées GPS invalides (valeurs NaN, coordonnées hors limites), et cas limites dans la sortie de détection (aucune détection, densité extrême). Trois mécanismes de défense sont intégrés au pipeline.

Validation des coordonnées GPS. Avant d'invoquer le filtre géospatial, chaque ligne du manifeste est validée : la latitude doit appartenir à l'intervalle $[-90^\circ, 90^\circ]$, la longitude à $[-180^\circ, 180^\circ]$, et aucune des deux valeurs ne peut être None ou NaN. Un échec de validation lève une exception typée (`GPSValidationError`), interceptée par le gestionnaire d'exceptions global, qui journalise l'anomalie dans le fichier `run_events.jsonl` et passe à l'image suivante sans interrompre le pipeline.

Détection d'anomalies image. Avant l'inférence ONNX, chaque tuile SAR est inspectée à la recherche de contenu dégénéré. Une tuile dont plus de 98% des pixels sont à zéro (entièrement noire) ou à 255 (entièrement saturée) est signalée comme anormale et rejetée sans inférence. Le seuil de 98% (plutôt que 99%) tient compte de la distribution d'intensité

naturellement clairsemée des scènes SAR de haute mer, où de vastes régions noires sont attendues et légitimes.

Gestionnaire d'exceptions global de sécurité. L'orchestrateur encapsule la boucle de traitement par image dans un gestionnaire d'exceptions générique. Toute exception non gérée — y compris une erreur de mémoire (`MemoryError`), une erreur d'exécution de la session ONNX, ou une erreur de format de données inattendue — est interceptée, journalisée avec la trace complète dans `run_events.jsonl`, puis suivie d'un passage immédiat à l'image suivante. Ce mécanisme garantit qu'une tuile corrompue isolée ne peut interrompre l'exécution d'une mission complète portant sur plus de 1000 images.

4. Chapitre 4 — Méthodologie expérimentale

Ce chapitre décrit le protocole expérimental complet mis en œuvre pour entraîner, compresser et évaluer le pipeline présenté au chapitre 3. Il couvre les jeux de données mobilisés, la procédure d'entraînement des modèles et sa reproductibilité, la méthodologie d'évaluation à deux niveaux rendue nécessaire par l'absence de métadonnées GPS dans les jeux de données SAR publics, les trois configurations expérimentales retenues, et le protocole de construction de la vérité terrain pour le taux de rétention des événements tactiques (TERR).

4.1 Jeux de données

4.1.1 Synthèse des jeux de données

Quatre jeux de données sont mobilisés à travers les phases d'entraînement, d'évaluation et de pré-entraînement optionnel. Le Tableau 5 en propose une synthèse ; les sections qui suivent détaillent le pipeline de préparation appliqué à chacun.

Tableau 5 : Jeux de données mobilisés dans ce travail

Jeu de données	Rôle	Images	Boîtes	Répartition (tr/val/te)	Propriétés clés
iVision-MRSSD	Entraînement principal (SAR)	11 590	27 891	9271/1158/1161	SAR multi-capteurs (ALOS-2, Sentinel-1, TerraSAR-X, ICEYE, Capella, PAZ). Inclut 1578 échantillons négatifs (mer vide).
Official-SSDD	Benchmark cross-domaine	1 160	2 587	Ensemble complet	SAR (Sentinel-1 / Gaofen-3). Utilisé uniquement pour l'évaluation de généralisation cross-domaine, jamais en entraînement.
SSDD_coco	Entraînement	1 160	2 587	928/116/116	Format masque polygonal COCO. Source : Official-

	segmentation				SSDD converti en annotations de segmentation COCO.
ship_dataset_v0	Pré-entraînement optionnel	17 826*	26 548	Tout en entraînement	Optique/SAR multi-capteurs (Gaofen, Sentinel). *21 903 images exclues par filtre qualité de 10 Ko.

Source : élaboration propre

Les effectifs d'images et de boîtes englobantes reflètent l'état post-préparation, après filtrage qualité et conversion de format. Le sous-ensemble de test iVision-MRSSD (1161 images) constitue le jeu d'évaluation principal pour les étages 2 à 5 du pipeline. Official-SSDD est utilisé exclusivement comme benchmark de généralisation cross-domaine.

4.1.2 Pipeline de préparation unifié

Les quatre jeux de données source se présentent sous des formats hétérogènes (XML Pascal VOC, JSON COCO, TXT YOLO). Un pipeline de préparation unifié les convertit vers un standard YOLO unique (un fichier d'étiquettes par image, coordonnées normalisées) avant toute étape d'entraînement ou d'évaluation. Ce pipeline applique successivement les opérations suivantes :

- conversion de format : les boîtes englobantes Pascal VOC ([xmin, ymin, xmax, ymax] en pixels) sont converties au format YOLO ([x_centre, y_centre, largeur, hauteur] normalisées dans [0, 1]) ; les masques polygonaux COCO sont convertis au format polygonal YOLO pour le jeu de données de segmentation ;
- normalisation sémantique : l'ensemble des classes source (ship, vessel, boat, liner, warship, tanker, etc.) est unifié en une seule classe d'identifiant 0 : navire. Ce choix de conception maximise la généralisation entre types de navires sans nécessiter d'annotation multi-classes, cohérent avec la tâche de détection binaire de l'étage 2 ;

- inclusion d'échantillons négatifs : 1578 images d'iVision-MRSSD ne contenant aucun navire (mer ouverte) sont incluses en entraînement avec des fichiers d'étiquettes vides. Ces échantillons négatifs ciblent spécifiquement la réduction des faux positifs issus d'artefacts de fouillis SAR tels que les motifs de déferlement de vagues et les reflets solaires ;
- filtrage qualité : 21 903 images de ship_dataset_v0 sont exclues par un filtre de taille de fichier minimale (seuil de 10 kilooctets). Ces images présentent une résolution spatiale insuffisante pour des caractéristiques exploitables au niveau du navire ;
- filtrage de corruption : 1441 fichiers d'annotation XML de la source Official-SSDD présentant une taille de zéro octet sont détectés et exclus.

La préparation est exécutée par le script `scripts/01_Data_Preparation/prepare_datasets.py` et produit un fichier `manifest.json` enregistrant les effectifs exacts d'images et de boîtes, les affectations de répartition, et les notes de traitement pour chaque jeu de données, assurant la traçabilité nécessaire à la reproductibilité.

4.2 Entraînement des modèles

4.2.1 Environnement d'entraînement

L'ensemble de l'entraînement a été réalisé sur Google Colab à l'aide d'instances GPU T4, avec PyTorch 2.5.1+cu124 et Ultralytics 8.3.110. L'entraînement a été conduit hors de l'environnement de simulation Docker, celui-ci étant exclusivement ONNX par conception. Le chemin d'entraînement vers déploiement est le suivant : affinage sur GPU (Colab) → export ONNX FP32 → quantification ONNX INT8 → déploiement Docker (inférence CPU). Pour pallier les interruptions de session Colab, une stratégie de sauvegarde incrémentale bidirectionnelle a été mise en œuvre : points de contrôle, fichiers CSV de résultats et arguments d'entraînement étaient écrits sur Google Drive toutes les 15 minutes, avec reprise automatique via l'option `resume=True` et le dernier point de contrôle sauvegardé (`last.pt`).

Un problème technique a été rencontré et résolu durant la phase d'export : Ultralytics 8.4.66 combiné à PyTorch 2.11.0 provoquait une erreur d'exécution lors de l'export ONNX de la tête de sortie unique du modèle de segmentation, du fait d'une incompatibilité dans le nombre de noms de sortie attendus. Ce problème a été résolu par un retour à la version Ultralytics 8.3.110 associée à PyTorch 2.5.1, sans aucune modification d'architecture.

4.2.2 Hyperparamètres

Le Tableau 6 recense les hyperparamètres utilisés pour le détecteur et le segmenteur, ainsi que la justification de chaque choix. Les décisions d'augmentation spécifiques à l'imagerie SAR y sont mises en évidence.

Tableau 6 : Hyperparamètres d'entraînement pour YOLOv8n (détecteur) et YOLOv8n-seg (segmenteur)

Hyperparamètre	Détecteur (MRSSD)	Segmenteur (SSDD)	Justification
Époques	100	80	Détecteur : convergence confirmée à l'époque 97 ; 80 retenu pour le segmenteur compte tenu d'un jeu de données plus restreint (1160 vs 11590 images).
Patience (arrêt anticipé)	20	15	Permet des gains tardifs (mAP@0.5:0.95 +0,0156 entre les époques 90 et 100) sans risque de surapprentissage.
Résolution d'entrée	640 × 640	640 × 640	Résolution standard YOLOv8 ; le remplissage carré préserve le ratio d'aspect des tuiles SAR.
HSV-H / HSV-S	0,0 / 0,0	0,0 / 0,0	L'imagerie SAR est fondée sur l'intensité ; les augmentations de teinte et de saturation sont physiquement dénuées de sens et corrompraient les

			statistiques de chatoiment (speckle).
HSV-V	0,30	0,30	Simule les fluctuations d'intensité SAR dues aux variations d'angle d'incidence et d'état de mer.
Mosaic	1,0	1,0	Déterminant pour la détection de petits navires dans de vastes étendues ; force le modèle à généraliser à travers l'échelle et le contexte.
Flip-H / Flip-V	0,5 / 0,3	0,5 / 0,3	Tient compte des directions de passage orbital variables (ascendant/descendant) — les navires apparaissent en miroir selon la géométrie d'acquisition SAR.
Taille de lot (batch)	16	8	Lot du segmenteur réduit en raison d'une consommation mémoire par échantillon plus élevée liée à la génération de masques.
Optimiseur	AdamW (auto)	AdamW (auto)	Réglage par défaut Ultralytics ; planification de taux d'apprentissage en cosinus avec préchauffage linéaire (3 époques).
Graine aléatoire	42	42	Fixée pour assurer la reproductibilité entre les sessions Colab et les exécutions locales.

Source : élaboration propre

Les augmentations spécifiques à l'imagerie SAR (HSV-H = 0, HSV-S = 0) reflètent la nature physique de cette imagerie en tant que carte d'intensité, et non image couleur.

4.2.3 Chronologie des sessions d'entraînement

L'entraînement a été réparti sur cinq sessions Colab distinctes, en raison des limitations de quota GPU et des délais d'expiration de session. Chaque session a produit des points de contrôle et des fichiers de résultats CSV conservés dans le dépôt du projet sous `notebooks/colab_runs/`. La session du 14 juin 2026 correspond à l'étude d'ablation décrite en section 4.4.

4.2.4 Export et quantification des modèles

À l'issue de l'entraînement, chaque modèle est exporté selon un pipeline de compression en deux étapes, implémenté dans `src/optimization/export_and_quantize.py` :

- étape 1 — export ONNX FP32 : le point de contrôle PyTorch (`best.pt`) est exporté au format ONNX en utilisant l'opset 17 avec simplification de graphe (`onnx-simplifier`). Le graphe résultant est indépendant du matériel et compatible avec ONNX Runtime sur tout processeur ARM ;
- étape 2 — quantification dynamique ONNX INT8 : la fonction `quantize_dynamic()` du module `onnxruntime.quantization` est appliquée au graphe FP32. Les poids sont quantifiés hors ligne en INT8 ; les échelles d'activation sont calculées au moment de l'inférence à partir de la distribution réelle des entrées. La quantification dynamique a été retenue plutôt que la quantification statique car elle ne nécessite aucun jeu de données de calibration, réduisant la complexité du pipeline tout en atteignant des compromis de précision acceptables pour le scénario de déploiement visé.

Les quatre artefacts de production résultants (détecteur FP32, détecteur INT8, segmenteur FP32, segmenteur INT8) sont stockés dans `models/optimized/` et constituent la chaîne d'inférence complète déployée dans l'environnement de simulation Docker.

4.3 Méthodologie d'évaluation à deux niveaux

4.3.1 Le problème de géolocalisation

Une caractéristique fondamentale de l'ensemble des jeux de données SAR de détection de navires mobilisés dans ce travail — iVision-MRSSD, Official-SSDD et ship_dataset_v0 — est que leurs images sont distribuées sous forme de vignettes JPEG extraites de scènes SAR plus vastes. Le processus de découpage en vignettes élimine les métadonnées de référencement spatial GeoTIFF d'origine, et les fichiers JPEG résultants ne portent aucune information GPS. Ce constat a été confirmé par un audit exhaustif des fichiers bruts des jeux de données : aucune balise GeoTIFF, aucun champ EXIF GPS, aucun fichier de coordonnées supplémentaire n'est présent.

Cette situation n'est pas propre à ce travail — il s'agit d'une caractéristique documentée de l'écosystème des jeux de données de détection de navires SAR, où le processus de découpage constitue une étape de prétraitement nécessaire à l'apprentissage supervisé mais élimine intrinsèquement les métadonnées spatiales. La conséquence pour ce pipeline est que l'étage 1 (filtre géospatial) et les étages 2 à 5 (pipeline ML) ne peuvent être conjointement évalués sur une même vérité terrain à l'aide de coordonnées réelles.

4.3.2 La solution à deux niveaux

Ce travail répond au problème de géolocalisation par une conception d'évaluation à deux niveaux, qui découple la correctitude géométrique de l'étage 1 de la performance d'apprentissage automatique des étages 2 à 5.

Niveau 1 — validation indépendante de l'étage 1. Le filtre géospatial (geo_filter.py) met en œuvre un algorithme déterministe d'intersection de polygones dont la correctitude est indépendante du contenu de l'image SAR. Sa validation ne nécessite pas d'images SAR réelles — elle nécessite des points géographiques de test vérifiés avec des classifications de zone connues. Une suite de 30 points de test géospatiaux synthétiques a été construite, chaque point ayant été vérifié manuellement contre les limites de Marine Regions v12. Six catégories sont couvertes : points côtiers Z1 près des ports marocains, points ZEE Z2, points de haute mer Z3, cas

limites à moins de 0,1° d'une frontière de zone, points terrestres, et points de la ZEE du Sahara occidental. Les résultats sont présentés au chapitre 5.

Niveau 2 — évaluation des étages 2 à 5 avec coordonnées synthétiques documentées. Chacune des 1161 images de test MRSSD se voit affecter une coordonnée maritime marocaine synthétique mais géographiquement valide, générée par échantillonnage aléatoire uniforme au sein du polygone de zone correspondant (Shapely, GeoJSON Marine Regions v12, graine = 42). L'affectation de coordonnées suit la distribution de couverture LEO réaliste documentée au Tableau 7.

Tableau 7 : Distribution par zone des coordonnées synthétiques du manifeste de test

Zone	Description	Images	Proportion	Justification opérationnelle
Z1	Eaux territoriales (12 MN)	116	10%	Priorité tactique maximale ; couverture spatiale la plus faible par passage orbital.
Z2	ZEE (au-delà de 12 MN)	348	30%	Zone de surveillance active ; fréquence de couverture intermédiaire.
Z3	Haute mer internationale	697	60%	Couverture LEO dominante ; priorité individuelle faible mais volume élevé.
Total	—	1 161	100%	Ensemble complet du test MRSSD (graine = 42)

Source : élaboration propre

La distribution 60/30/10 reflète les statistiques de couverture empiriques d'une orbite héliosynchrone au-dessus des zones maritimes marocaines : un satellite à 500 kilomètres d'altitude avec une largeur de fauchée de 60 kilomètres passe la majorité de chaque passage au-dessus de la haute mer internationale.

Note de transparence méthodologique : les coordonnées synthétiques permettent une évaluation complète du pipeline de bout en bout, incluant la classification de zone de l'étage 1 et la priorisation tactique de l'étage 3.

Elles n'impliquent pas que les navires détectés dans les images SAR soient physiquement situés aux positions affectées. L'évaluation du pipeline constitue donc une validation fonctionnelle de la logique de décision sous une distribution de zone réaliste, non une évaluation de précision de suivi géographique.

4.4 Configurations expérimentales

Trois configurations expérimentales sont évaluées afin de permettre une attribution claire des variations de performance à leur cause spécifique. Le Tableau 8 décrit chaque configuration, ses contraintes et son objectif de mesure.

Tableau 8 : Configurations expérimentales évaluées

Configuration	Format du modèle	Contraintes	Objectif de mesure
baseline	PyTorch FP32	Aucune (hôte non contraint)	Plafond de performance ML. Les métriques d'apprentissage (mAP, précision, rappel) proviennent de la validation Colab. Les métriques embarquées (latence, RAM) ne sont pas disponibles — l'orchestrateur est exclusivement ONNX par conception.
onnx_fp32	ONNX FP32	1 vCPU, 1 Go RAM	Isole l'impact des contraintes matérielles seules, indépendamment de la quantification. La comparaison avec baseline donne le surcoût des contraintes seules ; la comparaison avec onnx_int8 donne l'impact de la quantification seule.
onnx_int8	ONNX INT8	1 vCPU, 1 Go RAM	Simulation CubeSat principale. Combine contraintes matérielles et quantification INT8 telles qu'elles apparaîtraient sur un ordinateur de bord réel. Configuration de référence pour TERR, DRR et TIES.

Source : élaboration propre

Application des contraintes Docker. Les contraintes matérielles pour onnx_fp32 et onnx_int8 sont appliquées au niveau du conteneur via les limites de ressources docker-compose (cpus: '1', memory: '1g'). De plus, les variables d'environnement OMP_NUM_THREADS=1 et

ONNXRUNTIME_NUM_THREADS=1 sont définies pour empêcher ONNX Runtime de détecter et d'utiliser des cœurs CPU hôtes supplémentaires. Cette application multi-couche garantit que l'environnement simulé reflète fidèlement le profil mono-cœur et à mémoire contrainte de la classe de calculateur de bord ciblée.

Note sur la configuration baseline. L'orchestrateur de production est exclusivement ONNX par conception architecturale — PyTorch n'est pas installé dans le conteneur d'inférence. En conséquence, la configuration baseline ne peut être exécutée à travers le même pipeline Docker que onnx_fp32 et onnx_int8. Les métriques embarquées (latence, RAM, débit) pour baseline ne sont donc pas disponibles depuis la simulation Docker. Les métriques d'apprentissage automatique (mAP@0.5, précision, rappel) pour baseline sont issues de l'exécution de validation Colab et représentent le plafond de performance du modèle FP32 non contraint sur le jeu de validation intra-domaine.

4.5 Étude d'ablation : stratégie d'initialisation du segmenteur

La section 3.5 décrit deux stratégies d'initialisation pour le segmenteur : Configuration A (initialisation à chaud depuis l'ossature du détecteur SAR, yolov8n_mrssd_v1/weights/last.pt) et Configuration B (initialisation à chaud depuis des poids COCO génériques, yolov8n-seg.pt). Ces deux stratégies sont comparées par une étude d'ablation visant à déterminer si l'initialisation à chaud spécifique au domaine SAR apporte un avantage mesurable.

L'ablation est conçue pour isoler la variable d'initialisation en maintenant constants l'ensemble des autres hyperparamètres : les deux configurations utilisent le même jeu de données d'entraînement (SSDD_coco, 928 images), la même politique d'augmentation (Tableau 6), le même nombre d'époques (80), et la même graine aléatoire (42). Seule diffère la source des poids initiaux.

L'hypothèse nulle est que les deux configurations produisent une mAP@0.5 équivalente — c'est-à-dire que l'initialisation à chaud depuis le détecteur SAR n'apporte aucun avantage statistiquement significatif par rapport au pré-entraînement COCO. Si la Configuration A surpasse significativement la Configuration B, cela constituerait une preuve que le transfert intra-domaine (détection SAR vers segmentation SAR) est une technique pratique pour améliorer la convergence sous données limitées. Si les deux configurations sont équivalentes, cela suggérerait que la mAP@0.5 élevée observée (0,9800 pour la Configuration A) est attribuable à la relative simplicité du jeu de données SSDD_coco plutôt qu'à la stratégie d'initialisation. Les résultats de cette étude d'ablation sont rapportés au chapitre 5.

4.6 Construction de la vérité terrain pour TERR

Le calcul du taux de rétention des événements tactiques (TERR) nécessite un ensemble d'événements HIGH/CRITICAL de référence, contre lequel la sortie du pipeline est comparée. La méthodologie de construction est documentée au Tableau 9.

Tableau 9 : Méthodologie de construction de la vérité terrain pour TERR

Aspect	Ce travail	Implication pour TERR
Source de vérité terrain	Matrice de priorité appliquée par règles aux fichiers d'étiquettes YOLO (nombre de navires) × affectation de zone du manifeste	TERR mesure la cohérence interne du pipeline plutôt que l'accord avec le jugement indépendant d'un opérateur humain. C'est la validation de premier ordre appropriée pour un système de priorité déterministe.
Ensemble annoté	300 images issues de Z1 (116) et Z2 (184) — seules zones pouvant produire des événements HIGH/CRITICAL selon la matrice de priorité	Les images Z3 sont exclues du calcul de TERR ; elles ne peuvent produire d'événements HIGH/CRITICAL quel que soit le nombre de navires.
Nombre d'événements HC	47 images confirmées HIGH ou CRITICAL sur 300 annotées	Dénominateur de TERR = 47. Analyse de sensibilité : ± 1

	(15,7%)	événement correspond à $\pm 2,1\%$ de TERR, établissant le plancher de variance naturelle de la métrique à cette taille d'échantillon.
Reproductibilité	ground_truth_hc.json versionné dans le dépôt avec le champ ground_truth_method: rule_based_priority_matrix	Tout relecteur peut régénérer la vérité terrain à partir des étiquettes YOLO MRSSD et de test_manifest.csv à l'aide du script fourni.

Source : élaboration propre

Justification du périmètre d'annotation. Seules les images affectées à Z1 (116) ou Z2 (184) sont incluses dans l'ensemble d'annotation TERR, totalisant 300 images. Les images Z3 sont exclues car, selon la matrice de priorité (Tableau 2, section 3.4), la priorité maximale atteignable en Z3 est MEDIUM (rassemblement, plus de 4 navires) — les événements HIGH et CRITICAL sont structurellement impossibles en Z3, quel que soit le nombre de navires. Inclure les images Z3 dans l'ensemble d'annotation TERR gonflerait artificiellement le dénominateur avec des événements que le système classe correctement comme non critiques, produisant un dénominateur TERR trompeur.

Calcul de TERR par configuration. TERR est calculé séparément pour chaque configuration expérimentale (onnx_fp32 et onnx_int8) en exécutant le pipeline complet sur les 300 images annotées et en comparant les classifications de priorité résultantes à ground_truth_hc.json. Le TERR de la configuration baseline est défini à 1,000 par construction — la vérité terrain étant dérivée de la même matrice de priorité que celle utilisée par le pipeline, une implémentation baseline parfaite atteint trivialement $TERR = 1,0$. La comparaison significative est donc celle entre onnx_fp32 et onnx_int8, qui mesure si la quantification INT8 introduit des défaillances systématiques de détection sur les événements tactiquement critiques.

5. Chapitre 5 — Résultats

Ce chapitre présente les résultats expérimentaux obtenus selon trois niveaux de mesure : la performance d'apprentissage automatique des modèles de détection et de segmentation, la performance embarquée des trois configurations expérimentales sous contraintes matérielles Docker, et les métriques à l'échelle de la mission — DRR, TERR et TIES — qui évaluent conjointement la réduction de bande passante et la rétention de l'information tactique. Le symbole ★ désigne la meilleure valeur parmi les configurations comparables, et la mention N/A indique une métrique non disponible pour la configuration baseline en raison de la conception exclusivement ONNX de l'orchestrateur (chapitre 4).

5.1 Performance du détecteur

Le détecteur a été entraîné durant 100 époques sur le sous-ensemble d'entraînement iVision-MRSSD (9271 images). La convergence a été confirmée par la diminution simultanée des trois pertes de validation, sans divergence par rapport aux pertes d'entraînement correspondantes — signature caractéristique d'une généralisation réussie plutôt que d'un surapprentissage. Le meilleur point de contrôle a été sélectionné à l'époque 97, atteignant la mAP@0.5 maximale sur l'ensemble de l'exécution (0,9476). Le Tableau 10 détaille la trajectoire des métriques sur la fenêtre finale d'entraînement (époques 90 à 100).

Tableau 10 : Métriques de validation du détecteur sur la fenêtre finale d'entraînement

Métrique	Époque 90	Époque 97 (best)	Époque 100 (finale)	Interprétation
mAP@0.5	0,9462	0,9476 ★	0,9474	Maximum sur l'ensemble de l'exécution à l'époque 97.
mAP@0.5:0.95	0,5836	0,5982	0,5992 ★	+0,0156 entre les époques 90 et 100 ; gain le plus important de la fenêtre.
Précision	0,9078	0,9100 ★	0,9073	Environ 9% des détections sont de fausses alertes.
Rappel	0,8807	0,8849	0,8894 ★	Meilleur à l'époque 100 ; environ 11% des navires réels sont manqués.

Source : élaboration propre, données d'entraînement Colab

L'observation la plus significative du Tableau 10 concerne la trajectoire de la mAP@0.5:0.95, qui gagne 0,0156 point entre les époques 90 et 100 — le gain relatif le plus important de l'ensemble de la fenêtre d'entraînement, approximativement équivalent à la somme des gains cumulés entre les époques 10 et 90. Ce comportement tardif est attribuable à la planification du taux d'apprentissage en cosinus, qui conduit l'optimiseur vers un régime de faible bruit en fin d'entraînement, permettant un ajustement fin des boîtes englobantes sans perturber la classification déjà apprise à des taux d'apprentissage plus élevés.

5.2 Étude d'ablation du segmenteur

Le Tableau 11 compare les deux stratégies d'initialisation du segmenteur décrites au chapitre 4. La Configuration A initialise depuis l'ossature du détecteur entraîné sur MRSSD (initialisation à chaud

spécifique au domaine SAR) ; la Configuration B initialise depuis des poids génériques pré-entraînés sur COCO.

Tableau 11 : Étude d'ablation : stratégie d'initialisation du segmenteur

Configuration	Initialisation	mAP@0.5	Interprétation
Config. A — Segmenteur V1	Ossature YOLOv8n MRSSD (initialisation à chaud domaine SAR)	0,9800	Variante retenue en production.
Config. B — Segmenteur V2	Poids génériques COCO YOLOv8n- seg	0.9872	Contrôle de l'ablation.

Source : élaboration propre

La Configuration A atteint une mAP@0.5 de 0,9800, notablement supérieure à celle du détecteur (0,9474) malgré un jeu de données d'entraînement nettement plus restreint. Deux explications non exclusives sont cohérentes avec ce résultat : soit le jeu de données SSDD_coco présente une distribution plus homogène que MRSSD, constituant un défi de généralisation moindre, soit l'initialisation à chaud spécifique au domaine SAR fournit des représentations de caractéristiques déjà adaptées à l'intensité SAR, accélérant la convergence. La complétion de la Configuration B permettra de déterminer la contribution relative de ces deux facteurs.

5.3 Compression des modèles par quantification INT8

Le Tableau 12 rapporte la réduction de taille obtenue par quantification dynamique INT8 pour chaque modèle, ainsi que la perte de mAP@0.5 correspondante mesurée sur les jeux de validation respectifs.

Tableau 12 : Impact de la quantification INT8 sur la taille des modèles et la précision

Modèle	Taille FP32	Taille INT8	Réduction	Perte mAP@0.5
Détecteur (YOLOv8n MRSSD)	12,50 Mo	3,36 Mo	-46,3%	-0,84%
Segmenteur (YOLOv8n-seg SSDD)	12,50 Mo	3,69 Mo	-46,3%	≈ -0,50%
Pipeline total	25,00 Mo	7,05 Mo	-71,8%	< 1%

Source : élaboration propre

Les deux modèles atteignent une réduction de taille individuelle de 46,3%. L'empreinte totale du pipeline INT8, soit 7,05 mégaoctets pour les deux modèles combinés, s'inscrit confortablement dans la capacité de stockage flash typique d'un calculateur de bord de CubeSat (32 à 128 mégaoctets). La réduction de taille observée (facteur 1,79×) est inférieure au facteur théorique maximal de 4× attendu d'une conversion float32 vers int8, ce qui s'explique par la présence de données non pondérales dans les fichiers de modèle (structure de graphe, métadonnées) non quantifiées, et par le fait que la quantification dynamique ne cible par défaut que les opérateurs Linear et MatMul, laissant les activations de convolution en virgule flottante.

5.4 Performance embarquée sous contraintes CubeSat

Le Tableau 13 rapporte les métriques de performance embarquée — latence, RAM, débit et taille de modèle — pour les trois configurations expérimentales, mesurées sur l'ensemble complet du test MRSSD (1161 images).

Tableau 13 : Métriques de performance embarquée

Métrique	baseline	onnx_fp32	onnx_int8
Latence — moyenne (ms)	N/A *	219 ms	213 ms ★
Latence — P95 (ms)	N/A *	245 ms	292 ms
RAM pic (Mo)	> 1000 Mo *	268,8 Mo	252,4 Mo ★
Débit (im/s)	N/A *	4,00	4,03 ★
Taille modèle (Mo)	25,00 Mo	25,00 Mo	7,05 Mo ★

Source : élaboration propre. * Métriques baseline non disponibles : l'orchestrateur de production est exclusivement ONNX.

Inversion de latence sous INT8. De manière contre-intuitive, onnx_int8 atteint une latence moyenne légèrement inférieure (213 ms) à onnx_fp32 (219 ms). Ce résultat s'explique par un effet de composition du pipeline : la correction du fichier GeoJSON a introduit davantage d'images Z1/Z2 dans le pipeline actif, augmentant la proportion de tuiles déclenchant la segmentation. Le segmenteur INT8 étant plus rapide que le segmenteur FP32 sur ce chemin de traitement, INT8 bénéficie d'un avantage précisément sur les images où la segmentation est activée.

Marge mémoire disponible. Les deux configurations ONNX opèrent avec un pic de RAM inférieur à 269 mégaoctets, soit moins de 27% de la limite d'un gigaoctet imposée au conteneur. Cette marge substantielle (plus de 730 mégaoctets inutilisés) est significative sur le plan opérationnel : elle permet d'accueillir des processus concurrents du calculateur de bord (systèmes de télémétrie, contrôle d'attitude, gestion de l'alimentation) partageant le même bus mémoire sur un CubeSat réel, sans risque d'interruption du pipeline par saturation mémoire.

5.5 Validation du filtre géospatial

Le Tableau 14 rapporte les résultats de la suite de 30 points de test géospatiaux synthétiques décrite au chapitre 4. Chaque ligne correspond à

une catégorie de cas de test, avec la classification de zone attendue vérifiée manuellement contre les limites de Marine Regions v12.

Tableau 14 : Résultats de validation du filtre géospatial de l'étage 1

Catégorie	Tests	Cas	Réussis	Taux de réussite
Points côtiers Z1	T01-T05	5	4	80% — 1 cas limite (0,08° de la frontière 12 MN)
Points ZEE Z2	T06-T10	5	5	100%
Haute mer Z3	T11-T15	5	5	100%
Cas limites (< 0,1° frontière)	T16-T20	5	4	80% — incertitude de simplification polygonale
Points terrestres/hors zone	T21-T25	5	5	100%
ZEE Sahara occidental	T26-T30	5	5	100%
TOTAL	T01-T30	30	28	93,3% — les deux échecs sont des cas limites de frontière

Source : élaboration propre

Le taux de réussite global de 93,3% satisfait le seuil d'acceptation de plus de 90% fixé préalablement à l'évaluation. Les deux échecs concernent des points situés à moins de 0,1° d'une frontière de zone, directement attribuables à la simplification polygonale de 90 à 95% appliquée pour respecter la contrainte mémoire de l'environnement embarqué. À la latitude de la côte atlantique marocaine, 0,1° correspond approximativement à 9 à 11 kilomètres, ce qui reste inférieur à la largeur de 22,2 kilomètres de la zone des 12 milles nautiques elle-même. Sur le plan opérationnel, une erreur de classification à la frontière exacte correspond à un changement de priorité d'au plus un niveau, et non à un échec de détection du navire, celui-ci demeurant correctement identifié par l'étage 2 indépendamment de la classification de zone.

5.6 Résultats à l'échelle de la mission — DRR, TERR et TIES

Le Tableau 15 rapporte les trois métriques à l'échelle de la mission constituant le cadre d'évaluation TIES. Ces métriques sont évaluées sur l'ensemble complet du test MRSSD (1161 images) pour onnx_fp32 et onnx_int8, à l'aide du fichier de vérité terrain décrit au chapitre 4 (47 événements HC).

Tableau 15 : Métriques à l'échelle de la mission pour les trois configurations

Métrique	baseline	onnx_fp32	onnx_int8
DRR — empirique (x)	~155x †	154,43x	155,16x ★
DRR — opérationnel (x)	~1753x †	1753,5x	1753,5x
TERR	1,000 ‡	0,7872	0,8085 ★
TIES	—	121,57	125,45 ★
Événements HC retenus / total	47 / 47 ‡	37 / 47	38 / 47

Source : élaboration propre. † DRR baseline identique aux configurations ONNX (format de payload indépendant du modèle). ‡ TERR baseline = 1,000 par construction. HC = HIGH ou CRITICAL.

5.6.1 Taux de réduction volumétrique des données (DRR)

Deux valeurs de DRR sont rapportées pour chaque configuration. Le DRR empirique (155x) est mesuré sur les images de test MRSSD réelles, compressées au format JPEG avec une taille moyenne de 127,7 kilooctets. Le DRR opérationnel (environ 1753x) est calculé pour le scénario opérationnel où l'entrée brute est une tuile SAR float32 512×512 pixels d'environ 1 mégaoctet. L'écart entre ces deux valeurs, d'un facteur d'environ 11, reflète la compression JPEG déjà appliquée lors de la préparation du jeu de données — les vignettes MRSSD ne sont pas des données SAR float32 brutes. La quasi-identité du DRR entre onnx_fp32 et onnx_int8 confirme que le niveau de quantification n'affecte pas la construction du payload, celle-ci étant déterminée par le niveau de priorité et non par les poids du modèle.

5.6.2 Taux de rétention des événements tactiques (TERR)

TERR mesure la fraction des événements HIGH/CRITICAL de référence (47 événements) que le pipeline identifie correctement et achemine vers une transmission au Format B. La configuration onnx_int8 atteint $TERR = 0,8085$ (38 événements sur 47 retenus), contre $0,7872$ (37 sur 47) pour onnx_fp32. La différence de 2,13 points de pourcentage correspond exactement à un seul événement.

Interprétation de l'avantage TERR d'INT8. Sur une vérité terrain de 47 événements, un écart d'un seul événement correspond à une variance naturelle de $\pm 2,1\%$, exactement égale à l'écart observé. Cette différence se situe donc dans le plancher de variance de la métrique à cette taille d'échantillon, et ne doit pas être interprétée comme une preuve que la quantification INT8 améliore systématiquement le rappel tactique. L'interprétation correcte est que TERR demeure statistiquement stable entre les niveaux de quantification — ni onnx_fp32 ni onnx_int8 n'introduit de défaillance systématique de détection sur les événements HIGH/CRITICAL. C'est là la conclusion opérationnellement pertinente : la quantification ne crée pas de zone aveugle tactique.

5.6.3 Score d'efficacité du renseignement tactique (TIES)

$TIES = TERR \times DRR$ fournit un score composite capturant l'optimisation conjointe du rappel tactique et de la compression de bande passante. La configuration onnx_int8 atteint $TIES = 125,45$, contre $121,57$ pour onnx_fp32 — un avantage de 3,2% porté par le rappel légèrement supérieur (un événement additionnel retenu).

La configuration INT8 domine FP32 sur les cinq dimensions comparables : TERR supérieur, DRR supérieur (marginale), meilleur TIES, latence inférieure, RAM inférieure, et taille de modèle considérablement plus faible (-71,8%). Cette dominance monotone sur l'ensemble des métriques rend le choix de onnx_int8 comme configuration de déploiement recommandée sans ambiguïté. Le compromis lié au choix d'INT8 plutôt que FP32 — une perte de $mAP@0.5$ de 0,84% sur le détecteur — est largement compensé

par les avantages opérationnels dans un contexte de déploiement à bande passante contrainte.

5.7 Synthèse des résultats

La configuration `onnx_int8` atteint une performance supérieure ou équivalente sur chacune des métriques rapportées, comparée à `onnx_fp32`, tout en opérant sous des contraintes matérielles identiques. Les résultats clés sont les suivants :

- performance ML : $mAP@0.5 = 0,9474$ (détecteur), $mAP@0.5 = 0,9800$ (segmenteur), avec une dégradation inférieure à 1% de FP32 vers INT8 pour les deux modèles ;
- empreinte du modèle : 7,05 mégaoctets pour le pipeline total (INT8), soit une réduction de 71,8% par rapport aux 25,00 mégaoctets en FP32 ;
- performance embarquée : 213 ms de latence moyenne, 252,4 mégaoctets de RAM pic (moins de 27% de la limite d'un gigaoctet), débit de 4,03 images par seconde ;
- filtre géospatial : 93,3% de classification de zone correcte, les deux échecs se situant à des frontières de zone dans la zone d'incertitude de simplification polygonale ;
- métriques de mission : $DRR = 155\times$ empirique / $1753\times$ opérationnel, $TERR = 0,8085$, $TIES = 125,45$;
- dominance INT8 : `onnx_int8` surpasse `onnx_fp32` sur $TERR (+2,1\%)$, $TIES (+3,2\%)$, latence moyenne $(-2,7\%)$ et RAM $(-6,1\%)$, tout en réduisant la taille du modèle de 71,8%.

Ces résultats valident collectivement la thèse centrale de ce travail : un pipeline d'apprentissage automatique optimisé peut réduire la charge de communication d'un CubeSat de deux à trois ordres de grandeur, tout en préservant 80,9% des événements tactiquement critiques — et la quantification INT8, loin de dégrader cette capacité, la maintient avec des avantages opérationnels mesurables.

6. Chapitre 6 — Discussion

Ce chapitre interprète les résultats présentés au chapitre 5, discute leur portée et leurs limites, et examine les implications de ce travail pour une seconde phase de développement orientée vers un déploiement opérationnel.

6.1 Interprétation du résultat principal : la quantification INT8 ne crée pas de zone aveugle tactique

Le résultat empirique central de ce travail peut sembler simple à énoncer : la configuration INT8 surpasse la configuration FP32 sur chacune des métriques comparables — TERR, DRR, latence moyenne, RAM, débit et taille de modèle. La portée de ce résultat ne réside toutefois pas dans la supériorité d'INT8 en tant que telle, mais dans ce qu'il permet d'écarter. L'hypothèse de travail ayant motivé la conception expérimentale était la crainte que la quantification INT8 puisse introduire des zones aveugles tactiques systématiques — c'est-à-dire une tendance à manquer spécifiquement les événements HIGH ou CRITICAL que le modèle FP32 aurait détectés, créant une dégradation asymétrique précisément sur les événements les plus importants opérationnellement. Cette crainte est fondée : la quantification INT8 introduit des erreurs de représentation qui ne se distribuent pas uniformément sur l'espace des entrées, et il est plausible que les signatures de navires à faible contraste — précisément celles qui caractérisent les petits navires dissimulés dans des environnements côtiers encombrés Z1 — soient plus affectées que les signatures à fort contraste de haute mer.

La comparaison TERR (0,8085 pour INT8 contre 0,7872 pour FP32) constitue la principale preuve contre cette hypothèse pour la configuration de test actuelle. Ce résultat signifie que, sur les 47 événements HIGH/CRITICAL de référence du jeu de test MRSSD, le pipeline INT8 identifie correctement 38 événements (80,9%) et le pipeline FP32 en identifie correctement 37 (78,7%). La différence d'un seul événement se

situé dans le plancher de variance de $\pm 2,1\%$ imposé par la taille restreinte de la vérité terrain, et ne doit pas être interprétée comme un avantage statistiquement significatif d'INT8 sur le rappel tactique. L'interprétation correcte est que les deux configurations réalisent une performance équivalente sur le sous-ensemble d'événements critiques pour la mission — la quantification ne nuit pas sélectivement aux détections de priorité élevée.

Ce constat a une implication pratique directe pour la décision de déploiement sur CubeSat : à rappel tactique équivalent, INT8 doit être préféré sans condition, puisqu'il fournit un modèle 71,8% plus petit (7,05 contre 25,00 mégaoctets), une RAM pic inférieure de 6,1% (252,4 contre 268,8 mégaoctets), et une latence moyenne marginalement inférieure (213 contre 219 millisecondes). Le pipeline INT8 domine strictement FP32 au sens de Pareto — il est meilleur ou égal sur chaque dimension mesurée. Les 19,1% d'événements HIGH/CRITICAL restant manqués (9 pour INT8, 10 pour FP32) sont attribuables aux limites de rappel du détecteur en amont (88,9% de rappel global), et non à l'étape de quantification — une distinction qui a des implications directes sur les priorités d'amélioration futures de TIES.

6.2 TIES comme outil de navigation dans l'espace de conception

Au-delà de son rôle de métrique de résultat, $TIES = TERR \times DRR$ fournit une cible d'optimisation concrète pour les itérations futures de conception du pipeline. Le résultat actuel ($TIES = 125,45$ pour INT8) peut être décomposé en ses deux facteurs constitutifs afin d'identifier où des améliorations auraient le plus d'impact.

La composante DRR ($155,16\times$) est déjà très élevée au regard de toute contrainte de bande passante pratique. À un débit de liaison descendante en bande S de 1 mégabit par seconde, même un DRR de $50\times$ serait largement suffisant pour transmettre l'ensemble des événements dans une fenêtre de contact de six minutes. Faire passer le DRR de $155\times$ à

300× par une réduction supplémentaire de la taille des payloads n'ajouterait qu'une valeur opérationnelle marginale — la liaison n'est pas le facteur limitant. La priorité d'optimisation porte donc clairement sur TERR : améliorer TERR de 0,8085 à 0,9000, tout en maintenant un DRR supérieur ou égal à 150×, porterait TIES à environ 135, une amélioration de 7,6% reflétant une réduction réelle des événements critiques manqués plutôt qu'un artifice de compression.

Cette analyse révèle un principe de conception applicable au-delà de ce pipeline spécifique : pour la surveillance maritime par CubeSat, TIES est limité par le rappel du détecteur, non par la bande passante. Le pipeline comprime déjà les données de trois ordres de grandeur ; le facteur limitant est que 19% des événements critiques ne sont pas détectés par le modèle en amont. Les améliorations futures de TIES devraient donc privilégier les améliorations du détecteur (rappel plus élevé, affinage multi-capteurs, jeux d'entraînement plus vastes) plutôt que les optimisations de compression du payload. Il s'agit d'une conclusion non évidente que les métriques d'apprentissage automatique classiques seules — qui suggéreraient de continuer à optimiser la mAP — n'auraient pas révélée.

6.3 Priority Engine v2 : extensions architecturales et impact attendu sur TIES

Le moteur de priorité v2, implémenté après l'évaluation expérimentale principale, introduit deux modificateurs s'appliquant par-dessus la matrice de priorité de base (chapitre 3) : un modificateur de navire furtif et un modificateur de persistance orbitale. Ces extensions n'étaient pas actives lors des exécutions expérimentales ayant produit les résultats TERR, DRR et TIES rapportés au chapitre 5. Leur impact quantitatif sur TIES à grande échelle reste donc à mesurer, et une évaluation formelle est identifiée comme travail futur. Cette section propose une analyse qualitative de leurs effets attendus.

6.3.1 Modificateur de navire furtif

Le modificateur de navire furtif escalade la priorité des navires présentant une confiance de détection inférieure à 0,55 en zones Z1/Z2, sous l'hypothèse opérationnelle que les détections à faible confiance dans des zones sensibles sont davantage susceptibles de représenter des navires évitant activement l'immatriculation AIS que de véritables faux positifs. En l'absence de données AIS réelles permettant un recoupement, cette hypothèse ne peut être validée expérimentalement dans le cadre de ce travail — le modificateur opère sur un indicateur indirect (la confiance du détecteur) plutôt que sur une correspondance AIS avérée.

L'impact attendu sur TIES est ambigu. Si l'hypothèse est correcte, le modificateur augmente TERR en escaladant des événements que la matrice de base aurait classés LOW/MEDIUM vers HIGH/CRITICAL. Si l'hypothèse est incorrecte, le modificateur augmente le nombre de payloads Format B sans augmenter le nombre d'événements HC réels, gonflant artificiellement le dénominateur de bande passante transmise et réduisant ainsi le DRR. Ce compromis précision-rappel est intrinsèque à toute règle d'escalade fondée sur un seuil, et ne peut être quantifié qu'empiriquement, à l'aide d'une vérité terrain HC indépendante issue de données AIS réelles.

6.3.2 *Modificateur de persistance orbitale*

Le modificateur de persistance orbitale escalade la priorité des navires détectés dans la même cellule de grille spatiale de $0,1^\circ$ à travers plusieurs passages orbitaux successifs (minimum de 2 détections dans une fenêtre de 3 orbites). Ce modificateur est architecturalement plus robuste que le modificateur de navire furtif, car il repose sur une observation vérifiable — le navire se trouvait approximativement à la même position sur plusieurs passages satellitaires — plutôt que sur une inférence relative au statut d'immatriculation AIS.

La signification opérationnelle de l'escalade par persistance est bien établie en doctrine de surveillance maritime : un navire stationnant dans une zone sensible sur plusieurs heures (3 orbites LEO $\approx 4,7$ heures) sans

transiter constitue une cible de surveillance de priorité supérieure à un navire en transit, indépendamment de sa confiance de détection. L'impact attendu sur TIES est positif sous des hypothèses raisonnables de comportement des navires : la plupart des navires commerciaux légitimes sont en transit, tandis que les navires d'intérêt tendent à opérer de façon prolongée dans une zone restreinte. Pour le jeu de test MRSSD, constitué d'un instantané sans continuité temporelle, ce modificateur ne peut être correctement évalué — cela nécessite une séquence d'images ordonnée temporellement sur une même zone géographique, scénario que l'intégration de données Sentinel-1 réelles (Phase II) rendrait possible.

6.4 La simulation orbitale comme contribution méthodologique d'évaluation

Le cycle ACQUISITION/CONTACT_WINDOW/HANDOVER implémenté dans le gestionnaire de liaison descendante est présenté dans ce travail principalement comme un composant architectural permettant un flux de données réaliste entre le pipeline embarqué et le tableau de bord de la station sol. Sa contribution en tant que méthodologie d'évaluation mérite une discussion à part entière.

Les bancs d'essai classiques de l'IA embarquée évaluent la latence, le débit et la mémoire de façon isolée — ils répondent à la question « quelle est la vitesse d'exécution de ce modèle sur ce matériel ? ». La simulation orbitale permet une question différente et plus pertinente sur le plan opérationnel : « quelle proportion des événements détectés durant un passage orbital atteint effectivement la station sol, compte tenu de la durée de la fenêtre de contact, du débit de liaison descendante et de la distribution des priorités des événements détectés ? » Cette question ne peut être résolue par la seule mesure de latence ; elle nécessite de modéliser l'accumulation temporelle des événements dans le tampon embarqué et la transmission ordonnée par priorité durant la fenêtre de contact.

Un exemple concret illustre la valeur de cette approche. Dans la configuration expérimentale (fenêtre de contact de 6 minutes, bande S à 1 mégabit par seconde, environ 35% d'événements HIGH/CRITICAL parmi les détections), la bande passante de liaison descendante est effectivement surabondante par rapport à la sortie du pipeline : même à un débit soutenu de 4 images par seconde sur une fenêtre de contact de 6 minutes, le pipeline génère environ 1440 événements, dont environ 504 HIGH/CRITICAL et 936 LOW/MEDIUM, pour une charge utile totale d'environ 1,06 mégaoctet — largement en deçà de la capacité théorique de 450 mégaoctets d'une liaison de 6 minutes à 1 mégabit par seconde. Le DRR de 1753× rend la contrainte de bande passante effectivement non contraignante pour cette configuration de pipeline.

Cette conclusion — qu'au niveau de performance actuel du pipeline, le facteur limitant n'est pas la bande passante mais le rappel du détecteur — n'est visible qu'à travers le prisme de la simulation orbitale. Une évaluation purement fondée sur des bancs d'essai rapporterait la latence et le débit sans les relier à la question en aval de savoir si l'ensemble des événements tactiquement importants survit au trajet complet de l'orbite à la station sol.

6.5 Limites et conditions de validité des résultats

L'intégrité scientifique impose de reconnaître explicitement les conditions sous lesquelles les résultats rapportés sont valides, et celles sous lesquelles ils pourraient ne pas se transposer. Le Tableau 16 synthétise les six limites principales de ce travail, leur impact sur les résultats rapportés, et les corrections envisagées en Phase II.

Tableau 16 : Synthèse des limites principales de l'étude

Limite	Nature	Impact sur les résultats	Piste de correction (Phase II)
Coordonnées GPS synthétiques	Méthodologique — les coordonnées ne correspondent pas aux positions réelles des navires dans les tuiles SAR	Les affectations de zone sont réalistes en distribution mais non ancrées géographiquement. Les résultats des étages 1 et 3 sont valides comme validations fonctionnelles, non comme évaluations de suivi géographique.	Remplacer par les métadonnées GeoTIFF réelles de Sentinel-1.
Vérité terrain TERR circulaire	Méthodologique — la vérité terrain est dérivée de la même matrice de priorité que celle évaluée	TERR mesure la cohérence interne plutôt qu'un jugement humain indépendant. Le résultat de 0,8085 reflète le rappel de détection, non la correction de la classification du moteur de priorité lui-même.	Commissionner une annotation experte indépendante sur des scènes Sentinel-1 réelles.
Échantillon TERR restreint (N=47)	Statistique — plancher de variance de $\pm 2,1\%$ à cette taille d'échantillon	L'écart de TERR observé entre FP32 et INT8 (2,1%) se situe exactement dans le bruit statistique. Aucune différence statistiquement significative ne peut être revendiquée à N=47.	Étendre la vérité terrain à au moins 200 événements HC.
Métriques baseline embarquées non disponibles	Architecturale — l'orchestrateur exclusivement ONNX ne peut exécuter PyTorch	La comparaison de latence et de RAM ne couvre que onnx_fp32 vs onnx_int8. Le surcoût des contraintes seules	Implémenter un profilage PyTorch dans un conteneur séparé.

		(hôte → Docker) n'est pas directement mesuré.	
Images de test JPEG (non SAR float32 brut)	Liée au jeu de données — les vignettes MRSSD sont en JPEG, non en GeoTIFF float32	Le DRR empirique (155×) sous-estime significativement le DRR opérationnel (1753×). Les deux valeurs sont rapportées et l'écart est expliqué, mais le DRR empirique n'est pas le chiffre pertinent sur le plan opérationnel.	Évaluer sur des produits Sentinel-1 GRD bruts.
Contraintes Docker simulées vs. calculateur réel	Environnementale — les contraintes cgroups ne répliquent pas le comportement mémoire/cache d'un processeur ARM physique	Les mesures de latence reflètent la performance d'un hôte x86 contraint par Docker, non le comportement d'un processeur ARM Cortex-A. Les résultats sont indicatifs, non directement transposables à un calculateur de bord réel.	Portage sur Raspberry Pi CM4 (ARM Cortex-A72, RAM limitée à 1 Go).

Source : élaboration propre

Deux limites méritent une discussion complémentaire au-delà de ce que le Tableau 16 rapporte.

Sur la circularité de la vérité terrain fondée sur des règles. La vérité terrain TERR (ground_truth_hc.json) a été construite en appliquant la matrice de priorité aux étiquettes YOLO de MRSSD et aux affectations de zone synthétiques. Cela signifie que la vérité terrain et le pipeline partagent la même logique de priorité — un pipeline implémentant correctement la matrice atteint trivialement un TERR élevé sur cette vérité terrain, et les écarts par rapport à $TERR = 1,0$ ne reflètent que des échecs de détection, non des erreurs de classification du moteur de priorité lui-même. Il s'agit

d'une limite méthodologique divulguée de façon transparente, et non d'un défaut dissimulé. Le TERR tel qu'implémenté mesure : « parmi les événements que la matrice de priorité classerait comme HC si les détections étaient parfaites, combien le pipeline complet identifie-t-il correctement comme HC ? » Il s'agit d'une validation de premier ordre utile du pipeline de bout en bout, mais non équivalente à une évaluation validée par un opérateur humain quant aux événements réellement critiques sur le plan opérationnel.

Sur la simulation Docker par rapport à un ordinateur de bord réel. Les limites de ressources cgroups imposées par Docker contraignent l'allocation de l'ordonnanceur hôte au conteneur, mais ne répliquent pas l'architecture mémoire, le comportement de cache, ni les caractéristiques du jeu d'instructions d'un processeur ARM Cortex-A53 ou iMX8. Sur un ordinateur de bord réel, les optimisations NEON d'ONNX Runtime pour ARM pourraient produire des distributions de latence différentes de celles observées sur un hôte x86 ; la bande passante mémoire LPDDR4 diffère de la DDR4 ; et la limitation thermique sous charge d'inférence soutenue — préoccupation réelle pour un ordinateur de bord — n'a aucun équivalent dans un conteneur Docker. Les valeurs de latence rapportées (213 à 219 millisecondes en moyenne) doivent donc être interprétées comme indicatives du bon ordre de grandeur plutôt que comme des prédictions précises de la performance sur ordinateur de bord réel. Les résultats de RAM (252 à 269 mégaoctets) sont davantage transposables, le comportement d'allocation mémoire étant largement indépendant de l'architecture pour cette classe de charge de travail d'inférence.

6.6 Implications pour un déploiement opérationnel : vers la Phase II

Les résultats de la Phase I établissent que le pipeline d'inférence central — détection, segmentation conditionnelle, priorisation tactique et sérialisation efficace en bande passante — est fonctionnel, mesurable et optimisé pour la classe de contraintes ciblée. L'écart principal restant

entre la simulation actuelle et un système CubeSat opérationnel réside dans la source de données : la Phase I utilise un jeu de données statique en instantané avec des coordonnées synthétiques ; un système opérationnel doit traiter de l'imagerie Sentinel-1 en direct avec des coordonnées de scène réelles.

L'aptitude architecturale à cette transition est élevée. Le pipeline à cinq étages est agnostique à la source de données au niveau de l'orchestrateur — il consomme un manifeste de chemins d'images et de coordonnées GPS, indépendamment du fait que ces chemins pointent vers un jeu de données JPEG local ou vers des tuiles GeoTIFF fraîchement téléchargées depuis le portail d'accès libre Copernicus. Remplacer le chargeur de manifeste statique par un module de récupération Sentinel-1 constitue une modification localisée qui n'affecte ni les étages d'inférence, ni le sérialiseur, ni le tableau de bord.

Trois améliorations opérationnelles spécifiques sont immédiatement rendues possibles par l'intégration Sentinel-1. Premièrement, les coordonnées de scène réelles issues des métadonnées GeoTIFF éliminent l'affectation de coordonnées synthétiques, rendant l'évaluation du filtre géospatial (93,3% de précision, chapitre 5) directement applicable à des scénarios de surveillance authentiques plutôt qu'à des distributions de zone simulées. Deuxièmement, les tuiles SAR float32 (environ 1 mégaoctet par vignette 512×512) produisent un DRR empirique dans la fourchette de 1500 à 2000×, validant à l'échelle la valeur de DRR opérationnel. Troisièmement, des séquences d'images ordonnées temporellement sur une même zone géographique permettent au modificateur de persistance orbitale d'être correctement évalué et calibré contre des comportements réels de navires.

Au-delà de la source de données, l'ajout architectural principal que rend possible la Phase II est la transition d'une plateforme de simulation vers un service opérationnel interrogeable. La couche API REST envisagée (une application FastAPI encapsulant le pipeline existant) exposera deux points

de terminaison opérationnels : un point de terminaison de téléversement d'image pour l'analyse synchrone de tuiles SAR fournies par l'utilisateur, et un point de terminaison de requête géographique récupérant l'acquisition Sentinel-1 la plus récente disponible pour une zone d'intérêt spécifiée, et retournant la sortie du pipeline.

Conclusion générale

1. Synthèse des contributions

Ce rapport a présenté la conception, l'implémentation et l'évaluation d'un pipeline d'intelligence artificielle embarquée pour la surveillance maritime autonome à bord d'une plateforme CubeSat sous contraintes matérielles strictes. Ce travail apporte quatre contributions distinctes, présentées par ordre de nouveauté scientifique.

Premièrement, le cadre d'évaluation TIES. Ce travail introduit le taux de rétention des événements tactiques (TERR), le taux de réduction volumétrique des données (DRR), et leur produit, le score d'efficacité du renseignement tactique (TIES), en tant que cadre d'évaluation à l'échelle de la mission pour les pipelines de surveillance maritime embarqués. TIES répond à une lacune de la littérature existante : les métriques classiques d'apprentissage automatique mesurent la performance d'un modèle sur un jeu de test isolé, mais ne renseignent ni sur la criticité opérationnelle des détections manquées, ni sur l'adéquation des détections conservées avec les contraintes de communication d'une liaison descendante satellitaire. À notre connaissance, aucun cadre composite équivalent n'a été proposé spécifiquement pour le domaine de la surveillance maritime par imagerie SAR.

Deuxièmement, un pipeline conditionnel à cinq étages fonctionnant sous un gigaoctet de RAM. Ce travail démontre un pipeline complet de détection et de segmentation (filtre géospatial, détecteur YOLOv8n, moteur de priorité tactique, segmenteur conditionnel YOLOv8n-seg, sérialiseur JSON) opérant avec un pic de 252 mégaoctets de RAM sur un seul cœur de calcul, sans dépendance GPU. L'architecture conditionnelle réduit les FLOPs cumulés de segmentation d'environ 65% en n'activant le segmenteur que sur les événements de priorité HIGH/CRITICAL, soit environ 35% des tuiles traitées. L'empreinte totale du pipeline INT8, de

7,05 mégaoctets, s'inscrit dans la capacité de stockage flash de tout ordinateur de bord de CubeSat moderne.

Troisièmement, la validation empirique que la quantification INT8 ne crée pas de zone aveugle tactique. La configuration INT8 atteint $TERR = 0,8085$, statistiquement indiscernable de la référence FP32 ($TERR = 0,7872$, différence contenue dans le plancher de variance de $\pm 2,1\%$ à $N=47$), tout en offrant une réduction de taille de modèle de 71,8%. Ce résultat écarte la crainte que la quantification INT8 dégrade sélectivement la détection des événements de priorité élevée pour cette classe d'imagerie et d'architecture de modèle. La configuration INT8 domine strictement FP32 sur l'ensemble des dimensions mesurées.

Quatrièmement, un cadre de simulation orbitale pour l'évaluation de bout en bout du pipeline. Ce travail implémente un cycle paramétré ACQUISITION/CONTACT_WINDOW/HANDOVER couplé à un tampon de liaison descendante conscient des priorités et à un tableau de bord de station sol en temps réel, permettant une simulation de bout en bout du trajet de communication satellite-vers-sol. Ce cadre révèle qu'au niveau de performance actuel du pipeline, le facteur limitant est le rappel du détecteur, et non la bande passante — une conclusion invisible sans le contexte orbital.

2. Travaux futurs

Les résultats de la Phase I établissent une fondation validée pour trois directions de travaux futurs, organisées par priorité et dépendance au Tableau 17.

Tableau 17 : Feuille de route des travaux futurs, organisée par priorité

Priorité	Direction	Impact attendu sur TIES	Dépendance
1 — Haute	Intégration GeoTIFF Sentinel-1 via l'API Copernicus — remplace les coordonnées synthétiques par des métadonnées de scène réelles	Permet un DRR empirique réel (~1500-2000x) et une vérité terrain TERR indépendante via recoupement AIS	Pipeline de données Phase II (module de récupération Sentinel-1)
1 — Haute	Extension de la vérité terrain HC à au moins 200 événements — réduit le plancher de variance de TERR de $\pm 2,1\%$ à moins de $\pm 1\%$	Permet une comparaison TERR FP32 vs INT8 statistiquement significative	Annotation manuelle ou recoupement AIS sur scènes Sentinel-1
2 — Moyenne	Caractérisation mathématique formelle de TIES — bornage, monotonie, analyse de sensibilité, comparaison avec alternatives additives	Renforce TIES comme cadre de métrique publiable, indépendant de ce pipeline spécifique	Analyse théorique ; aucune expérimentation additionnelle requise
2 — Moyenne	Évaluation quantitative du moteur de priorité v2 (modificateurs navire furtif et persistance orbitale) sur l'ensemble du jeu de test	Première mesure empirique de l'effet de l'escalade par navire furtif sur TIES	Données Sentinel-1 réelles avec vérité terrain AIS
2 — Moyenne	Validation sur calculateur de bord physique (Raspberry Pi CM4, ARM Cortex-A72, RAM limitée à 1 Go)	Remplace les estimations de latence Docker par des mesures d'inférence ARM réelles	Acquisition matérielle ; portage ONNX Runtime ARM
3 — Basse	Classification multi-classes des navires (cargo, pétrolier, pêche, militaire) — nouveau modèle à entraîner	Permet des règles de priorité spécifiques au type de navire	Nouveau jeu de données annoté par type de navire

Parmi les directions de Priorité 1, l'intégration Sentinel-1 constitue l'étape unique la plus impactante. Elle résout simultanément quatre des six limites identifiées au chapitre 6 : les coordonnées de scène réelles remplacent les affectations synthétiques, les tuiles float32 produisent des valeurs de DRR opérationnellement représentatives, les acquisitions ordonnées temporellement permettent au modificateur de persistance orbitale d'être correctement évalué, et la capacité de recoupement AIS disponible via Copernicus fournit une vérité terrain indépendante pour TERR. La transition d'une évaluation sur jeu de données statique vers une évaluation sur données en direct constitue l'étape architecturale qui transforme le pipeline d'une preuve de concept vers un système opérationnel fonctionnel.

La caractérisation mathématique formelle de TIES (Priorité 2) est identifiée comme le prérequis pour une éventuelle publication à comité de lecture de ce cadre. La formulation actuelle ($TIES = TERR \times DRR$) est motivée sur le plan opérationnel — la forme multiplicative encode l'exigence que le rappel tactique et la réduction de bande passante constituent tous deux des conditions nécessaires — mais une publication rigoureuse nécessiterait une analyse du bornage, de la monotonie, de la sensibilité aux variations de composantes individuelles, et une comparaison avec des formulations composites alternatives.

3. Mot de conclusion

La thèse centrale de ce travail est que le plafond de performance en apprentissage automatique d'un modèle de détection ne constitue pas la contrainte déterminante pour un pipeline de surveillance maritime embarqué sur CubeSat — la contrainte déterminante est la combinaison du rappel du détecteur, de la durée de la fenêtre de contact, et de l'exigence de transmettre une information tactiquement significative plutôt qu'une imagerie brute. Le cadre TIES rend cette contrainte explicite et mesurable. Les résultats expérimentaux confirment que la

quantification INT8, qui réduit le modèle à 7,05 mégaoctets et opère dans 252 mégaoctets de RAM, atteint un TIES essentiellement équivalent à la référence FP32 non contrainte — suggérant que l'effort d'optimisation des itérations futures devrait se concentrer en amont, sur la qualité des données d'entraînement et le rappel de détection, plutôt qu'en aval, sur une compression supplémentaire du modèle.

Ces conclusions, issues d'une simulation complète du cycle opérationnel d'un CubeSat, sont directement exploitables pour la conception de futures missions de surveillance maritime. Une constellation de CubeSats équipés du pipeline INT8 décrit dans ce travail, recevant l'imagerie SAR Sentinel-1 au-dessus des eaux territoriales et de la zone économique exclusive marocaines, pourrait détecter, classer et prioriser de manière autonome l'activité des navires, ne transmettant que le sous-ensemble tactiquement pertinent des observations lors de chaque fenêtre de contact. Le score TIES fournit un indicateur unique d'efficacité de mission, permettant de comparer des variantes de pipeline, de guider les itérations de conception, et de communiquer la performance à des parties prenantes de la connaissance du domaine maritime qui ne sont pas spécialistes de l'apprentissage automatique.

L'ensemble du pipeline, des modèles entraînés, des scripts d'évaluation et du tableau de bord de station sol est disponible publiquement sous licence libre. Ce travail constitue une fondation validée pour la Phase II, dont l'objectif est de transformer cette preuve de concept en une plateforme opérationnelle exploitant des données satellitaires réelles.

Bibliographie

Les références suivantes sont classées par ordre alphabétique du nom de famille du premier auteur, conformément au style auteur-date (norme APA, 7ème édition). Chaque référence correspond à une citation effectivement mobilisée dans le corps du rapport (chapitres 2 et 6).

Articles scientifiques

- Denby, B., & Lucia, B. (2020). *Orbital edge computing: Nanosatellite constellations as a new class of computer system*. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2020)* (p. 505-520). Association for Computing Machinery. <https://doi.org/10.1145/3373376.3378473>
- Furano, G., Meoni, G., Dunne, A., Moloney, D., Ferlet-Cavrois, V., Tavoularis, A., Byrne, J., Buckley, L., Psarakis, M., Voss, K.-O., & Fanucci, L. (2022). Towards the use of artificial intelligence on the edge in space systems: Challenges and opportunities. *IEEE Aerospace and Electronic Systems Magazine*, 35(12), 44-56. <https://doi.org/10.1109/MAES.2020.3008468>
- Giuffrida, G., Fanucci, L., Meoni, G., Batič, M., Buckley, L., Dunne, A., van Dijk, C., Esposito, M., Hefele, J., Vercruyssen, N., Furano, G., Pastena, M., & Aubrun, J. (2021). The Φ -Sat-1 mission: The first on-board deep neural network demonstrator for satellite Earth observation. *IEEE Transactions on Geoscience and Remote Sensing*, 60, 1-14. <https://doi.org/10.1109/TGRS.2021.3125567>
- Hu, J., Zhi, X., Zhang, W., Ren, L., & Bruzzone, L. (2020). Superpixel-based subspace low-rank learning method for hyperspectral denoising. *IEEE Access*, 8, 105826-105840. <https://doi.org/10.1109/ACCESS.2020.2999713>
- Huang, L., Liu, B., Li, B., Guo, W., Yu, W., Zhang, Z., & Yu, W. (2023). A lightweight YOLOv8 network for real-time SAR ship detection on multi-sensor imagery. *Remote Sensing*, 15(18), 4551. <https://doi.org/10.3390/rs15184551>
- Li, J., Qu, C., & Shao, J. (2017). Ship detection in SAR images based on an improved faster R-CNN. In *2017 SAR in Big Data Era: Models, Methods and Applications (BIGSAR DATA)* (p. 1-6). IEEE. <https://doi.org/10.1109/BIGSAR DATA.2017.8124934>
- Li, K., Zhang, M., Xu, M., Tang, R., Wang, L., & Wang, H. (2022). Ship detection in SAR images based on feature enhancement Swin transformer and adjacent feature fusion. *Remote Sensing*, 14(13), 3186. <https://doi.org/10.3390/rs14133186>
- Liu, S., Kong, W., Chen, X., Xu, M., Yasir, M., Zhao, L., & Li, J. (2022). Multi-scale ship detection algorithm based on a lightweight neural network for spaceborne SAR images. *Remote Sensing*, 14(5), 1149. <https://doi.org/10.3390/rs14051149>
- Wang, Y., Wang, C., Zhang, H., Dong, Y., & Wei, S. (2019). A SAR dataset of ship detection for deep learning under complex backgrounds. *Remote Sensing*, 11(7), 765. <https://doi.org/10.3390/rs11070765>

Zhao, K., Zhou, Y., & Chen, X. (2022). A dense connection based SAR ship detection network with attention mechanism. *IEEE Access*, 10, 112796–112805. <https://doi.org/10.1109/ACCESS.2022.3216670>

Rapports et documents institutionnels

Flanders Marine Institute (VLIZ). (2023). *Maritime boundaries geodatabase: Maritime boundaries and exclusive economic zones (200NM), version 12*. <https://doi.org/10.14284/632>

Sites internet (webographie)

Ultralytics. (2024). *YOLOv8 documentation*. <https://docs.ultralytics.com>. Consulté le 10 juin 2026.

ONNX Runtime. (2024). *ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator*. <https://onnxruntime.ai>. Consulté le 12 juin 2026.

Copernicus Open Access Hub. (2026). *Sentinel-1 mission overview*. European Space Agency. <https://scihub.copernicus.eu>. Consulté le 20 juin 2026.

Annexes

Les annexes suivantes complètent le corps du rapport. Elles rassemblent des éléments de référence (liste des tableaux, cartographie logicielle, schémas de payload, environnement de simulation, formalisation des métriques de mission et matrices opérationnelles) destinés à faciliter la lecture, la reproductibilité et l'exploitation ultérieure du travail.

Annexe A — Liste des tableaux

Les tableaux ci-dessous apparaissent dans le corps du rapport, numérotés dans l'ordre de première mention.

- Tableau 1 — Taxonomie des zones du filtre géospatial (Chap. 3)
- Tableau 2 — Matrice de priorité tactique à deux dimensions (Chap. 3)
- Tableau 3 — Formats de payload JSON produits par le sérialiseur (Chap. 3)
- Tableau 4 — Décisions architecturales majeures et alternatives écartées (Chap. 3)
- Tableau 5 — Jeux de données mobilisés dans ce travail (Chap. 4)
- Tableau 6 — Hyperparamètres d'entraînement YOLOv8n / YOLOv8n-seg (Chap. 4)
- Tableau 7 — Distribution par zone des coordonnées synthétiques du manifeste de test (Chap. 4)
- Tableau 8 — Configurations expérimentales évaluées (Chap. 4)
- Tableau 9 — Méthodologie de construction de la vérité terrain pour TERR (Chap. 4)
- Tableau 10 — Métriques de validation du détecteur (fenêtre finale d'entraînement) (Chap. 5)
- Tableau 11 — Étude d'ablation : stratégie d'initialisation du segmenteur (Chap. 5)
- Tableau 12 — Impact de la quantification INT8 sur la taille et la précision (Chap. 5)
- Tableau 13 — Métriques de performance embarquée (Chap. 5)
- Tableau 14 — Résultats de validation du filtre géospatial (Chap. 5)
- Tableau 15 — Métriques à l'échelle de la mission (DRR, TERR, TIES) (Chap. 5)

Tableau 16 — Synthèse des limites principales de l'étude (Chap. 6)

Tableau 17 — Feuille de route des travaux futurs (Conclusion)

Note : aucune figure numérotée n'est référencée de façon formelle dans les chapitres actuels ; les éventuelles figures ajoutées lors de l'assemblage final pourront faire l'objet d'une liste dédiée en Annexe A bis.

Annexe B — Architecture logicielle et modules du dépôt

Le code source du projet maritime-edge-ai est organisé de manière à séparer le pipeline d'inférence embarqué, l'entraînement, l'évaluation, la télémétrie et les services Docker. La structure ci-dessous reflète l'arborescence du dépôt au moment de la rédaction du rapport.

B.1 Pipeline embarqué (src/pipeline/)

orchestrator.py — orchestration de la chaîne de décision à cinq étages
geo_filter.py — étage 1 : filtre géospatial (intersection GeoJSON / ZEE)
detector.py — étage 2 : détection de navires (ONNX Runtime, YOLOv8n)
priority_engine.py — étage 3 : moteur de priorité tactique (zone x densité)
segmenter.py — étage 4 : segmentation d'instance conditionnelle
serializer.py — étage 5 : sérialisation JSON atomique (Formats A / B)

B.2 Entraînement et optimisation

src/training/train_detector.py — entraînement du détecteur YOLOv8n (MRSSD)
src/training/train_segmenter.py — entraînement du segmenteur YOLOv8n-seg (SSDD)
src/optimization/export_and_quantize.py — export ONNX et quantification dynamique INT8
src/datasets/pipeline.py — préparation unifiée des jeux de données
src/datasets/converters.py — conversions d'annotations (VOC / COCO)

B.3 Évaluation et télémétrie

src/evaluation/ml_metrics.py — métriques d'apprentissage (mAP, précision, rappel)
src/evaluation/edge_metrics.py — métriques embarquées (latence, RAM, débit)
src/evaluation/mission_metrics.py — métriques de mission (DRR, TERR, TIES)
src/telemetry/orbital_simulator.py — simulation de fenêtres de contact orbital
src/telemetry/downlink_manager.py — gestion de la liaison descendante
src/telemetry/contact_window_executor.py — exécution lors des fenêtres de contact

B.4 Services et simulation

services/ (geo-filter, detector, priority-engine, segmenter, serializer, orchestrator)

docker/ + docker-compose.yml — conteneurs cubesat-core et ground-dashboard
dashboard/ — application Streamlit de station sol

L'ensemble du pipeline, des modèles entraînés, des scripts d'évaluation et du tableau de bord de station sol est mis à disposition sous licence libre, conformément à l'engagement de reproductibilité formulé en conclusion.

Annexe C — Formats de payload JSON (Formats A et B)

Le sérialiseur (serializer.py) produit deux variantes de payload, sélectionnées selon le niveau de priorité tactique attribué par le moteur de priorité. Les tailles indiquées correspondent à des mesures sur des payloads représentatifs (cf. Tableau 3 du chapitre 3).

C.1 Format A — transmission différée (priorités LOW / MEDIUM)

Taille typique : environ 598 octets. Champs inclus :

- identifiant d'événement ; identifiant d'image ;
- coordonnées GPS (latitude, longitude) ; zone géographique (Z1 / Z2 / Z3) ;
- niveau de priorité ; nombre de navires détectés ;
- horodatage ; latence du détecteur ; version du modèle ; raison tactique.

Mode de transmission : mise en tampon et envoi lors de la prochaine fenêtre de contact orbital.

C.2 Format B — transmission prioritaire (priorités HIGH / CRITICAL)

Taille typique : environ 994 octets. Champs inclus :

- tous les champs du Format A ;
- boîtes englobantes par navire ;
- contour du masque de segmentation ; surface en pixels ;
- orientation ; ratio d'aspect ; latence du segmenteur.

Mode de transmission : envoi prioritaire, sans mise en attente.

C.3 Écriture atomique

Afin d'éviter les lectures partielles côté station sol (JSONDecodeError), chaque payload est d'abord écrit dans un fichier temporaire

(payload.json.tmp), puis finalisé par `os.replace()` — opération atomique au sens POSIX sur le volume Docker partagé.

C.4 Schéma conceptuel (champs principaux)

```
{
  "event_id": "...",
  "image_id": "...",
  "gps": { "lat": 0.0, "lon": 0.0 },
  "zone": "Z1|Z2|Z3",
  "priority": "LOW|MEDIUM|HIGH|CRITICAL",
  "n_ships": 0,
  "timestamp": "...",
  "detector_latency_ms": 0,
  "model_version": "...",
  "tactical_reason": "...",
  // Format B uniquement :
  "ships": [ { "bbox": [...], "mask": [...], "area_px": 0,
               "orientation": 0, "aspect_ratio": 0 } ],
  "segmenter_latency_ms": 0
}
```

Annexe D — Environnement de simulation Docker

Le système de bout en bout est conteneurisé selon une architecture à deux services, simulant la séparation physique entre le calculateur de bord et le terminal de réception au sol (cf. section 3.7).

D.1 Conteneur cubesat-core (inférence embarquée)

- Image de base : `python:3.10-slim` ;
- Contraintes : 1 vCPU, plafond de 1 Go de RAM ;
- `OMP_NUM_THREADS = 1` ; `ONNXRUNTIME_NUM_THREADS = 1` ;
- Inférence exclusivement via ONNX Runtime (CPU) — PyTorch exclu du conteneur ;
- Exécute le pipeline complet à cinq étages.

D.2 Conteneur ground-dashboard (station sol)

- Application Streamlit de visualisation ;
- Vues : moniteur temps réel (état du pipeline, tuile annotée, matrice de décision) ; analyse de mission (répartition des priorités, métriques agrégées) ;

- Lecture des payloads via volume Docker partagé (écriture atomique côté cubesat-core).

D.3 Configurations expérimentales associées

Trois configurations sont évaluées dans le rapport (Tableau 8) : baseline PyTorch FP32 (hôte non contraint), onnx_fp32 (1 vCPU / 1 Go) et onnx_int8 (1 vCPU / 1 Go). Les métriques de mission du Tableau 15 sont calculées sur les configurations ONNX sous contraintes Docker.

Annexe E — Formules des métriques de mission (DRR, TERR, TIES)

Le cadre d'évaluation orienté mission introduit dans ce travail combine un indicateur de compression de bande passante et un indicateur de rétention des événements tactiquement critiques. Les définitions ci-dessous synthétisent les formalisations utilisées aux chapitres 4 et 5.

E.1 DRR — Data Reduction Ratio (taux de réduction volumétrique)

Le DRR mesure le facteur de réduction entre le volume des données d'entrée et le volume des payloads effectivement transmis :

$$\text{DRR} = \text{Volume_entrée} / \text{Volume_payloads_transmis}$$

Deux variantes sont rapportées : (i) DRR empirique, mesuré sur les vignettes JPEG du jeu de test MRSSD (taille moyenne $\approx 127,7$ ko) ; (ii) DRR opérationnel, estimé pour des tuiles SAR float32 512×512 (≈ 1 Mo). Valeurs de référence (configuration onnx_int8) : DRR empirique $\approx 155\times$; DRR opérationnel $\approx 1\,753\times$.

E.2 TERR — Tactical Event Recall Rate (taux de rétention des événements tactiques)

Le TERR mesure la fraction des événements de priorité HIGH ou CRITICAL (HC) de la vérité terrain qui sont correctement identifiés et acheminés en Format B :

$$\text{TERR} = N_{\text{événements_HC_retenus}} / N_{\text{événements_HC_référence}}$$

Sur la vérité terrain construite pour ce travail ($N = 47$ événements HC), la configuration onnx_int8 atteint $\text{TERR} = 0,8085$ (38/47), et onnx_fp32 atteint $\text{TERR} = 0,7872$ (37/47). La baseline de référence a $\text{TERR} = 1,000$ par construction.

E.3 TIES — Tactical Intelligence Efficiency Score

Le score composite d'efficacité du renseignement tactique est défini par le produit :

$$\text{TIES} = \text{TERR} \times \text{DRR}$$

Il capture conjointement le rappel des événements critiques et la compression de bande passante. Valeurs de référence (DRR empirique) : $\text{TIES}(\text{onnx_int8}) = 125,45$; $\text{TIES}(\text{onnx_fp32}) = 121,57$. L'interprétation opérationnelle de ces scores est discutée au chapitre 6.

E.4 Précautions d'interprétation

- Sur $N = 47$, un écart d'un seul événement HC correspond à une variance d'environ $\pm 2,1$ % sur TERR ; les petites différences FP32/INT8 ne doivent pas être sur-interprétées.
- Le DRR est indépendant du niveau de quantification des poids : il dépend du format de payload et de la distribution des priorités.
- La vérité terrain TERR de cette étude est dérivée de la matrice de priorité du système ; une annotation experte indépendante (Phase II, recoupement AIS / Sentinel-1) est requise pour une validation externe.

Annexe F — Taxonomie des zones et matrice de priorité tactique

Cette annexe rappelle, sous forme condensée, les règles opérationnelles du filtre géospatial et du moteur de priorité (Tableaux 1 et 2 du chapitre 3), utiles comme aide-mémoire hors du fil principal de lecture.

F.1 Taxonomie des zones (filtre géospatial)

- Z0 — Non-maritime : hors ZEE → rejet immédiat (aucune inférence)
- Z1 — Eaux territoriales : 0-12 MN de la ligne de base → sensibilité max.
- Z2 — Zone économique excl. : au-delà de 12 MN (ZEE) → sensibilité moyenne
- Z3 — Haute mer : au-delà de la ZEE → sensibilité minimale

Sources géographiques : polygones Marine Regions (VLIZ), y compris la façade marocaine et le Sahara occidental pour la délimitation ZEE retenue dans ce travail.

F.2 Seuils de déclenchement de la segmentation

- Z1 : segmentation dès 1 navire détecté
- Z2 : segmentation dès 2 navires détectés

Z3 : segmentation dès 5 navires détectés

La segmentation (étage 4) n'est active que pour les événements escaladés en HIGH/CRITICAL (Condition 4 du modèle de latence par morceaux). Environ 35 % des tuiles du jeu de test déclenchent cette condition, soit une réduction d'environ 65 % des FLOPs cumulés de segmentation.

F.3 Matrice de priorité (zone × densité de navires)

La priorité est une fonction de la zone géographique et du nombre de navires détectés (isolé, groupe réduit, groupe dense). Les niveaux possibles sont LOW, MEDIUM, HIGH et CRITICAL. Seuls HIGH et CRITICAL produisent un payload Format B et activent le segmenteur. Les détails cellulaires de la matrice figurent au Tableau 2 ; les extensions prévues en Priority Engine v2 (modificateur « navire furtif », modificateur de persistance orbitale) sont discutées en section 6.3.

Fin des annexes. Pour toute reproduction expérimentale, se référer au dépôt logiciel, aux scripts d'évaluation (src/evaluation/) et aux configurations Docker fournies avec le projet.

Table des matières

Remerciements	
Liste des abréviations	
Introduction générale	1
 Chapitre 1 — Contexte et problématique	3
1.1 La surveillance maritime et l'opportunité des nanosatellites.....	3
1.2 La contrainte fondamentale : la fenêtre de contact orbital.....	3
1.3 Formulation de la problématique.....	4
1.3.1 Détection sous contrainte de ressources.....	5
1.3.2 Priorisation tactique de l'information.....	5
1.3.3 Évaluation à l'échelle de la mission.....	6
1.4 Objectifs du travail.....	6
 Chapitre 2 — État de l'art	8
2.1 Détection de navires en imagerie SAR.....	8
2.2 Intelligence artificielle embarquée sur plateformes spatiales.....	9
2.3 Compression de modèles pour l'inférence sous contrainte.....	11
2.4 Cadres d'évaluation orientés mission.....	12
2.5 Positionnement du présent travail.....	13
 Chapitre 3 — Architecture du système	14
3.1 Philosophie de conception : chaîne de décision plutôt que traitement par lot.....	14
3.2 Étage 1 — Filtre géospatial (geo_filter.py).....	15
3.3 Étage 2 — Détection des navires (detector.py).....	17
3.4 Étage 3 — Moteur de priorité tactique (priority_engine.py).....	18
3.5 Étage 4 — Segmentation d'instance conditionnelle (segmenter.py).....	20
3.6 Étage 5 — Sérialisation de la télémétrie (serializer.py).....	21
3.7 Environnement de simulation Docker.....	23
3.8 Synthèse des décisions de conception majeures.....	24
3.9 Conception défensive : validation des entrées et mécanismes de sécurité.....	25
 Chapitre 4 — Méthodologie expérimentale	27

4.1 Jeux de données.....	27
4.1.1 Synthèse des jeux de données.....	27
4.1.2 Pipeline de préparation unifié.....	28
4.2 Entraînement des modèles.....	29
4.2.1 Environnement d'entraînement.....	29
4.2.2 Hyperparamètres.....	30
4.2.3 Chronologie des sessions d'entraînement.....	31
4.2.4 Export et quantification des modèles.....	32
4.3 Méthodologie d'évaluation à deux niveaux.....	32
4.3.1 Le problème de géolocalisation.....	32
4.3.2 La solution à deux niveaux.....	33
4.4 Configurations expérimentales.....	35
4.5 Étude d'ablation : stratégie d'initialisation du segmenteur.....	36
4.6 Construction de la vérité terrain pour TERR.....	37
Chapitre 5 — Résultats.....	39
5.1 Performance du détecteur.....	39
5.2 Étude d'ablation du segmenteur.....	40
5.3 Compression des modèles par quantification INT8.....	41
5.4 Performance embarquée sous contraintes CubeSat.....	42
5.5 Validation du filtre géospatial.....	43
5.6 Résultats à l'échelle de la mission — DRR, TERR et TIES.....	45
5.6.1 Taux de réduction volumétrique des données (DRR).....	45
5.6.2 Taux de rétention des événements tactiques (TERR).....	46
5.6.3 Score d'efficacité du renseignement tactique (TIES).....	46
5.7 Synthèse des résultats.....	47
Chapitre 6 — Discussion.....	48
6.1 Interprétation du résultat principal : la quantification INT8 ne crée pas de zone aveugle tactique.....	48
6.2 TIES comme outil de navigation dans l'espace de conception.....	49
6.3 Priority Engine v2 : extensions architecturales et impact attendu sur TIES.....	50
6.3.1 Modificateur de navire furtif.....	50
6.3.2 Modificateur de persistance orbitale.....	51
6.4 La simulation orbitale comme contribution méthodologique d'évaluation.....	52

6.5 Limites et conditions de validité des résultats.....	53
6.6 Implications pour un déploiement opérationnel : vers la Phase II.....	56
Conclusion générale	59
1. Synthèse des contributions.....	59
2. Travaux futurs.....	60
3. Mot de conclusion.....	62
Bibliographie	64
Articles scientifiques.....	64
Rapports et documents institutionnels.....	65
Sites internet (webographie).....	65
Annexes	66
Annexe A — Liste des tableaux.....	66
Annexe B — Architecture logicielle et modules du dépôt.....	67
Annexe C — Formats de payload JSON (Formats A et B).....	68
Annexe D — Environnement de simulation Docker.....	69
Annexe E — Formules des métriques de mission (DRR, TERR, TIES).....	70
Annexe F — Taxonomie des zones et matrice de priorité tactique.....	71